



Konfigurationsmanagement

Christian Macho

Konfigurationsmanagement



Motivation

- Softwaresysteme **ändern** sich fortlaufend
 - **Korrektur** von Fehlern
 - **Anforderungen** ändern sich
 - **Systemumgebung** ändert sich (Hardware, Betriebssystem, etc.)
- Änderungen führen zu einer **neuen Version** einer Software
 - **Welche Version** korrigierte diesen Fehler, **welche Version** implementierte das neue Feature?



Motivation

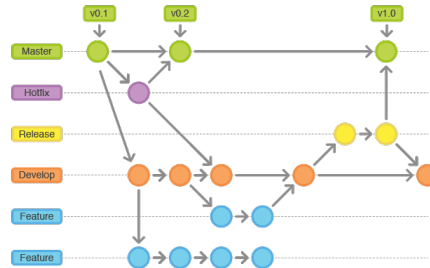
- **Software Configuration Management (SCM)**
 - betrifft die Anwendung (und Entwicklung) von Verfahren (Standards), Prozessen und Tools zur Verwaltung sich weiterentwickelnder Systemprodukte (=Änderungen) [4, p. 651]
 - Manchmal ist SCM auch Teil des Qualitäts-Managements
 - Dem Prozess sollten Standards zugrundeliegen (z.B.: IEEE 828-1990 [3])
- **SCM unterstützt das Entwickeln** von Systemen in Teams
 - Teammitglieder wissen über die Änderungen Bescheid und können Überschneidungen vermeiden

Aktivitäten/Themen



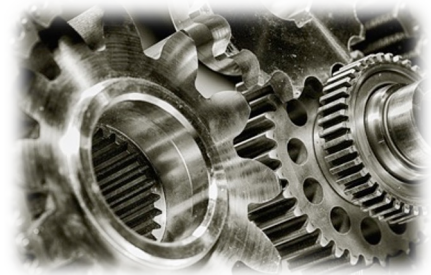
Änderungsmanagement

- Erfassen der Änderungen und Abschätzen der Realisierbarkeit und Kosten
- *Wie gehen SE Prozesse mit Änderungen um?*



Versionsmanagement

- Verwalten der verschiedenen Versionen der System-Komponenten und Vermeidung von Überschneidungen
- *Wie verwaltet man Versionen und Änderungen im Source Code effizient?*

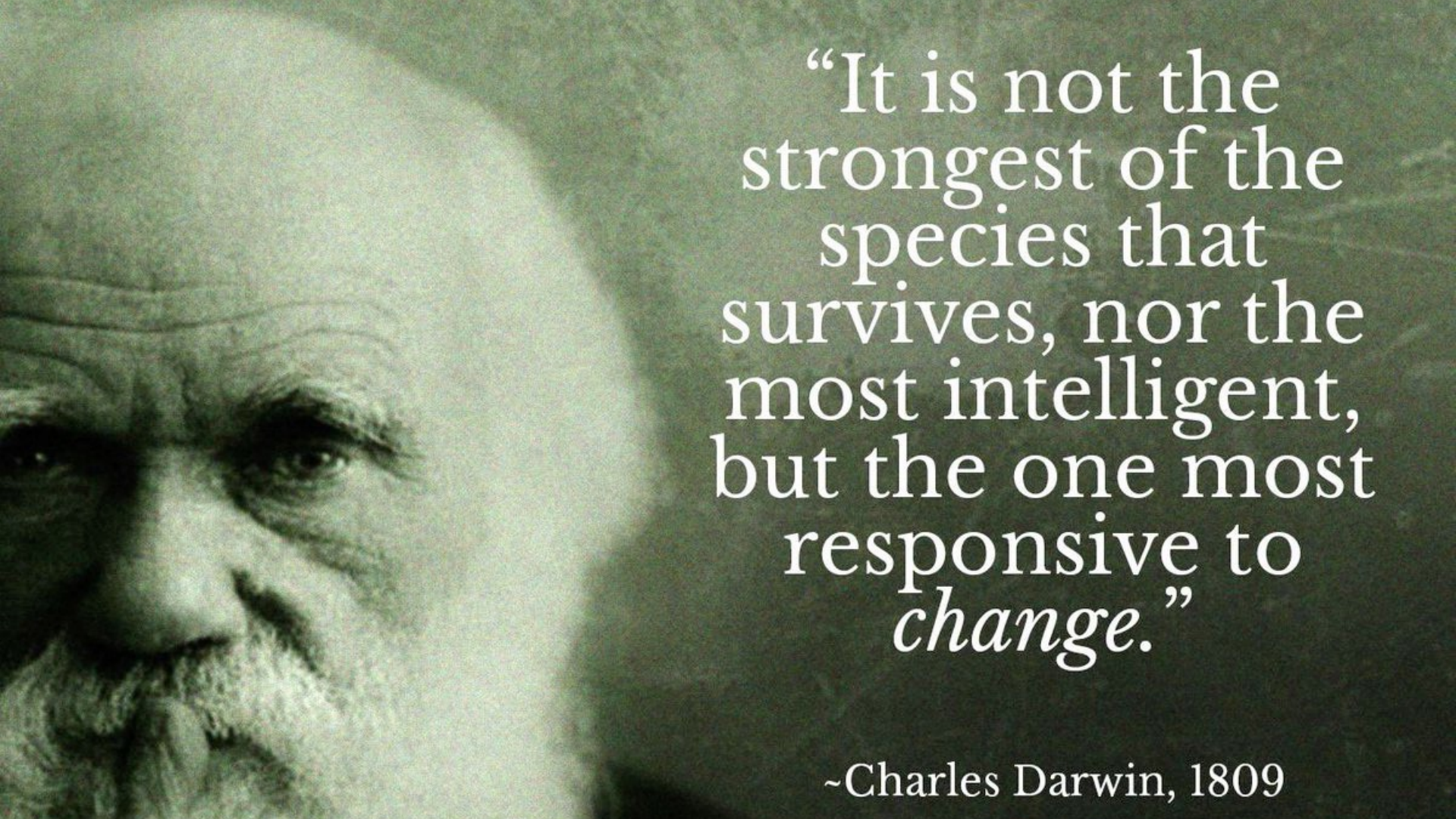


Automatisierung

- Build Management
- Continuous Integration/Delivery/Deployment
- DevOps
- *Wie kann man die Softwareausbringung automatisieren?*



ÄNDERUNGSMANAGEMENT



“It is not the
strongest of the
species that
survives, nor the
most intelligent,
but the one most
responsive to
change.”

~Charles Darwin, 1809

Änderungsmanagement, klassische Sicht

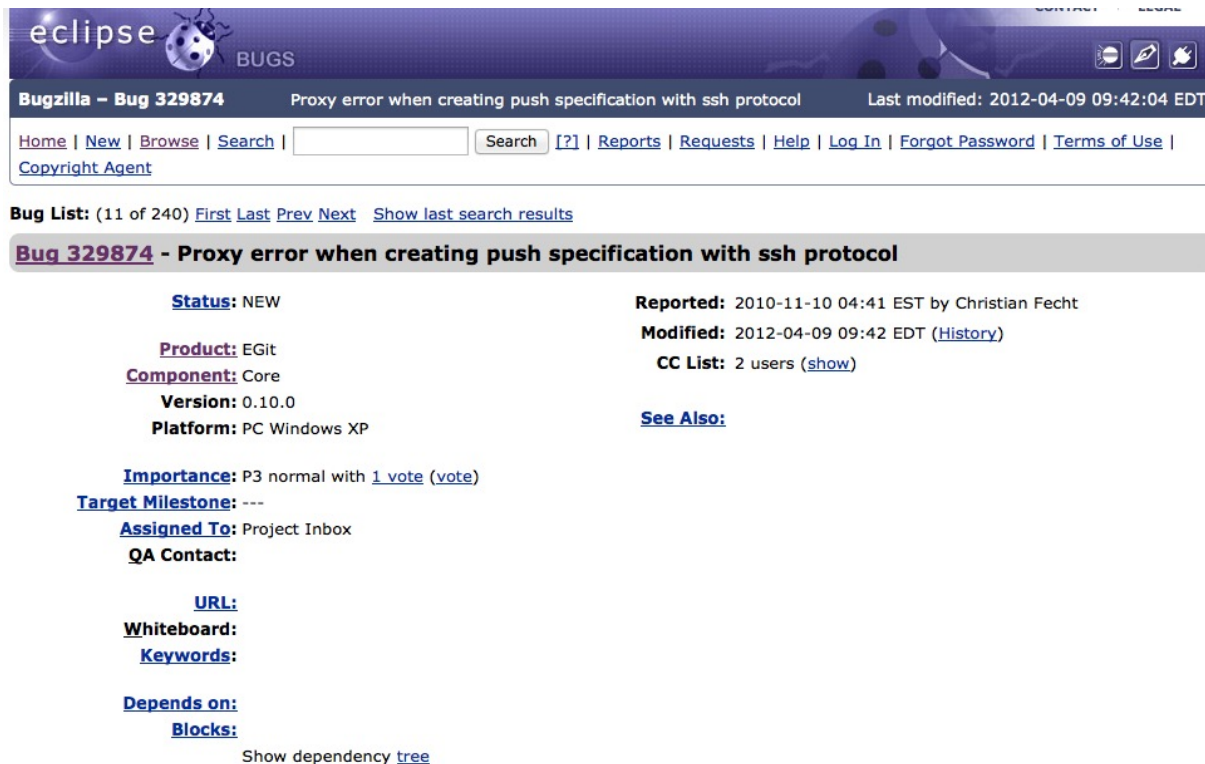
- Änderungen müssen in kontrollierter Weise (und dadurch kostengünstig) ablaufen [4, p. 657]:

```
Änderungsantrag stellen;  
Analyse des Änderungsantrags;  
if (Änderung notwendig) {  
    Bewertung, wie Änderung implementiert werden könnte;  
    Einschätzung der Änderungskosten;  
    Speichern des Änderungsantrags in der DB;  
    Einreichen des Antrags an Änderungskommission;  
    if (Änderung akzeptiert) {  
        do {  
            Durchführen der Änderung an der SW;  
            Speichern der Änderung unter Bezug zum Antrag;  
            Geänderte SW zum Test weiterleiten;  
        } while (Softwarequalität noch nicht hinreichend)  
    } else {  
        Änderungsantrag ablehnen;  
    }  
} else {  
    Änderungsantrag ablehnen;  
}
```


Änderungskommission

- Beurteilt die Auswirkung von Änderungen aus **strategischer** und **organisatorischer** Sicht (und nicht so sehr aus techn. Sicht)
 - Was sind die Konsequenzen, wenn diese Änderung nicht gemacht wird?
 - Was ist der zu erwartende Nutzen der Änderung?
 - Wie viele Benutzer sind von der Änderung betroffen?
 - Was sind die Kosten der Durchführung der Änderung?
 - Was ist der Releaseplan?
 - Soll die Änderung noch für die nächste Release durchgeführt werden?
- Agile Methoden involvieren den Kunden in diesen Entscheidungsprozess

Beispiel – Eclipse Bug Report



The screenshot shows the Eclipse Bugzilla interface. At the top, there's a header with the Eclipse logo and the word 'BUGS'. Below this, a navigation bar contains the text 'Bugzilla – Bug 329874', the title 'Proxy error when creating push specification with ssh protocol', and the date 'Last modified: 2012-04-09 09:42:04 EDT'. A search bar is also present. The main content area displays the bug details for 'Bug 329874 - Proxy error when creating push specification with ssh protocol'. It includes fields for Status (NEW), Product (EGit), Component (Core), Version (0.10.0), and Platform (PC Windows XP). It also shows the date and time it was reported (2010-11-10 04:41 EST by Christian Fecht), the date it was modified (2012-04-09 09:42 EDT), and the number of users affected (2 users). There are links for 'See Also', 'Importance', 'Target Milestone', 'Assigned To', 'QA Contact', 'URL', 'Whiteboard', 'Keywords', 'Depends on', and 'Blocks'. At the bottom, there is a link to 'Show dependency tree'.

Bugzilla – Bug 329874 Proxy error when creating push specification with ssh protocol Last modified: 2012-04-09 09:42:04 EDT

[Home](#) | [New](#) | [Browse](#) | [Search](#) | [\[?\]](#) | [Reports](#) | [Requests](#) | [Help](#) | [Log In](#) | [Forgot Password](#) | [Terms of Use](#) | [Copyright Agent](#)

Bug List: (11 of 240) [First](#) [Last](#) [Prev](#) [Next](#) [Show last search results](#)

Bug 329874 - Proxy error when creating push specification with ssh protocol

Status: NEW

Product: EGit

Component: Core

Version: 0.10.0

Platform: PC Windows XP

Reported: 2010-11-10 04:41 EST by Christian Fecht

Modified: 2012-04-09 09:42 EDT ([History](#))

CC List: 2 users ([show](#))

[See Also:](#)

Importance: P3 normal with [1 vote](#) ([vote](#))

Target Milestone: ---

Assigned To: Project Inbox

QA Contact:

URL:

Whiteboard:

Keywords:

Depends on:

Blocks:

Show dependency [tree](#)

Beispiel – Eclipse Bug Report

Christian Fecht 2010-11-10 04:41:48 EST

[Description](#)

Hi all,

I get an unexpected proxy error when creating a push specification using the ssh protocol. This proxy error seems only to occur if and only if a proxy is configured for HTTPS. Since proxies should only affect the HTTP / HTTPS protocol, but not the socket-based SSH protocol, I consider this a bug.

Error Message:

ProxyHTTP:java.ioException: proxy error: Forbidden#

My Software:

- o Eclipse Helios SR1
- o Eclipse EGit (Incubation) 0.10.0.201011011856
- o Eclipse JGit (Incubation) 0.10.0.201011011852

How to reproduce?

- o Configure a proxy server for HTTPS in Preferences -> Network Connections.
- o Create a push specification using ssh as protocol.
- o You should now see the above error message.
- o Remove the proxy for HTTP.
- o Try again to create a push specification using ssh.
- o It should now work.

Kind Regards,
Christian

Beispiel – Eclipse Bug Report

Christian Fecht 2010-11-10 04:41:48 EST

[Description](#)

Hi all,

I get an unexpected proxy error when creating a push specification using the ssh protocol. This proxy error seems only to occur if and only if a proxy is configured for HTTPS. Since proxies should only affect the HTTP / HTTPS protocol, but not the socket-based SSH protocol, I consider this a bug.

Für welche Softwareentwicklungsprozesse klappt diese Vorgehensweise gut?

o Eclipse EGit (Incubation) 0.10.0.201011011856
o Eclipse JGit (Incubation) 0.10.0.201011011852

How to reproduce?

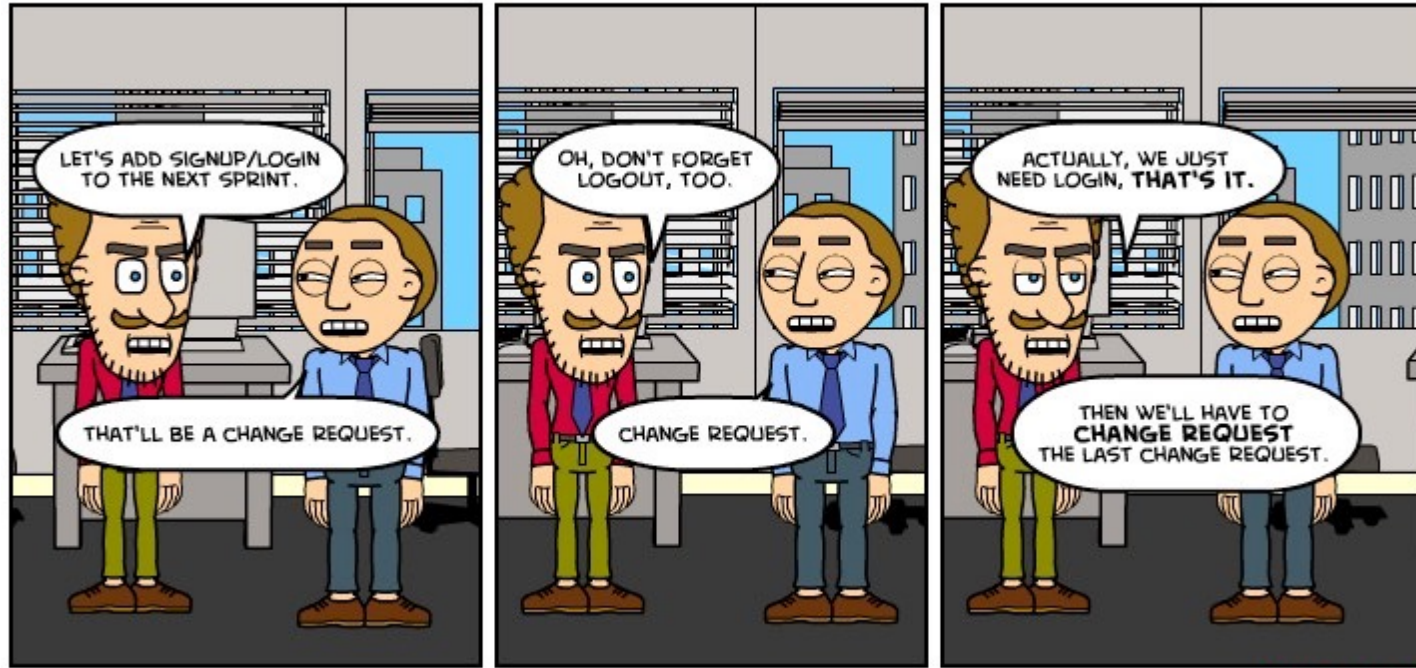
- o Configure a proxy server for HTTPS in Preferences -> Network Connections.
- o Create a push specification using ssh as protocol.
- o You should now see the above error message.
- o Remove the proxy for HTTP.
- o Try again to create a push specification using ssh.
- o It should now work.

Kind Regards,
Christian

What about Agile?

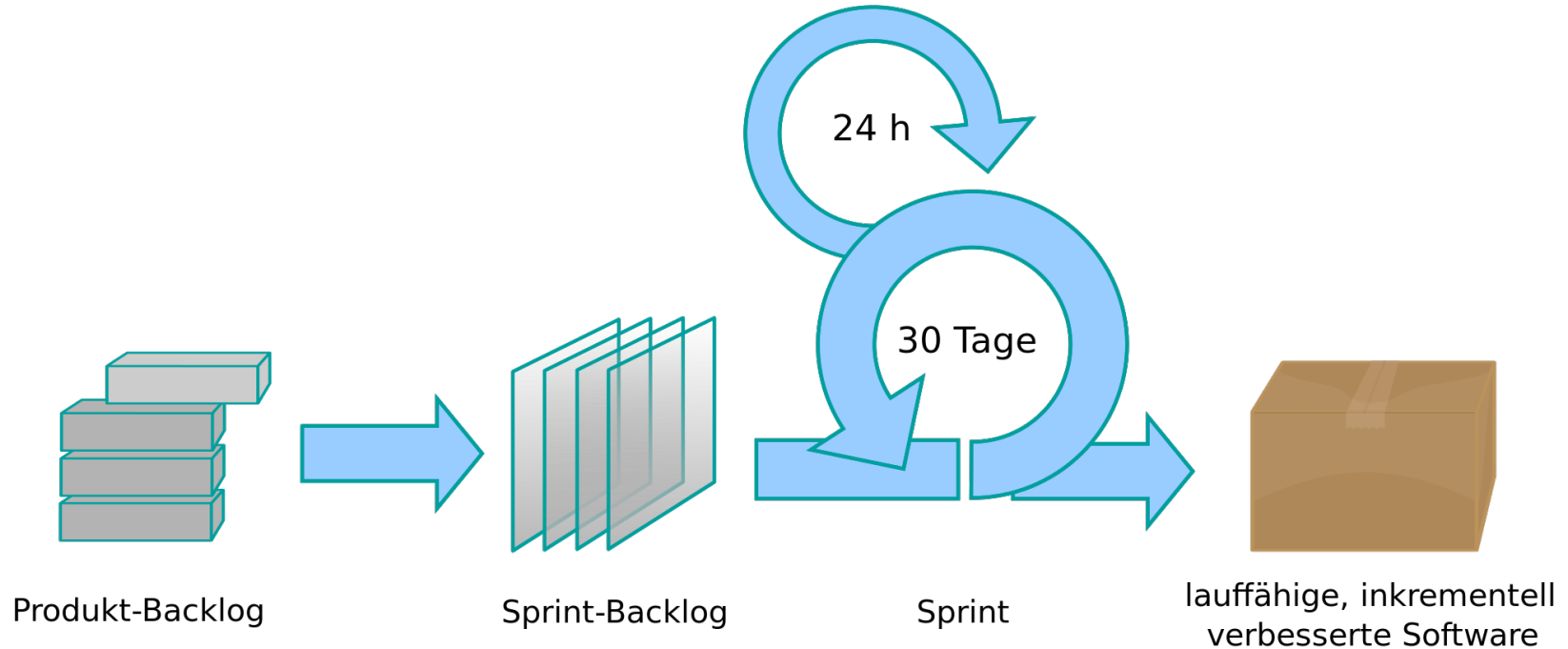
CHANGE REQUEST

BY RYAN CHARTRAND



X-TEAM.COM

Änderungsmanagement, agile Sicht

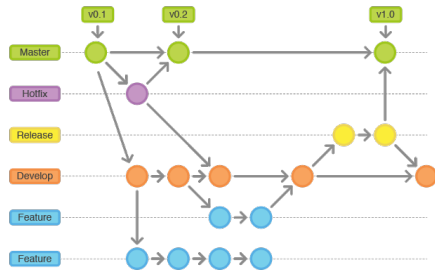


Änderungsmanagement, agile Sicht

- Änderungsanforderungen werden wie “gewöhnliche” Anforderungen behandelt
- Als User Story initial in das Produkt – Backlog
- Planung der Umsetzung je nach Priorität der Anforderung in einem Sprint
- Umsetzung der Anforderung im Sprint
- Auslieferung nach Sprintende
- **Was passiert wenn die Änderung sofort durchgeführt werden muss? (vgl. auch Produktionssupport)**

Zusammenfassung

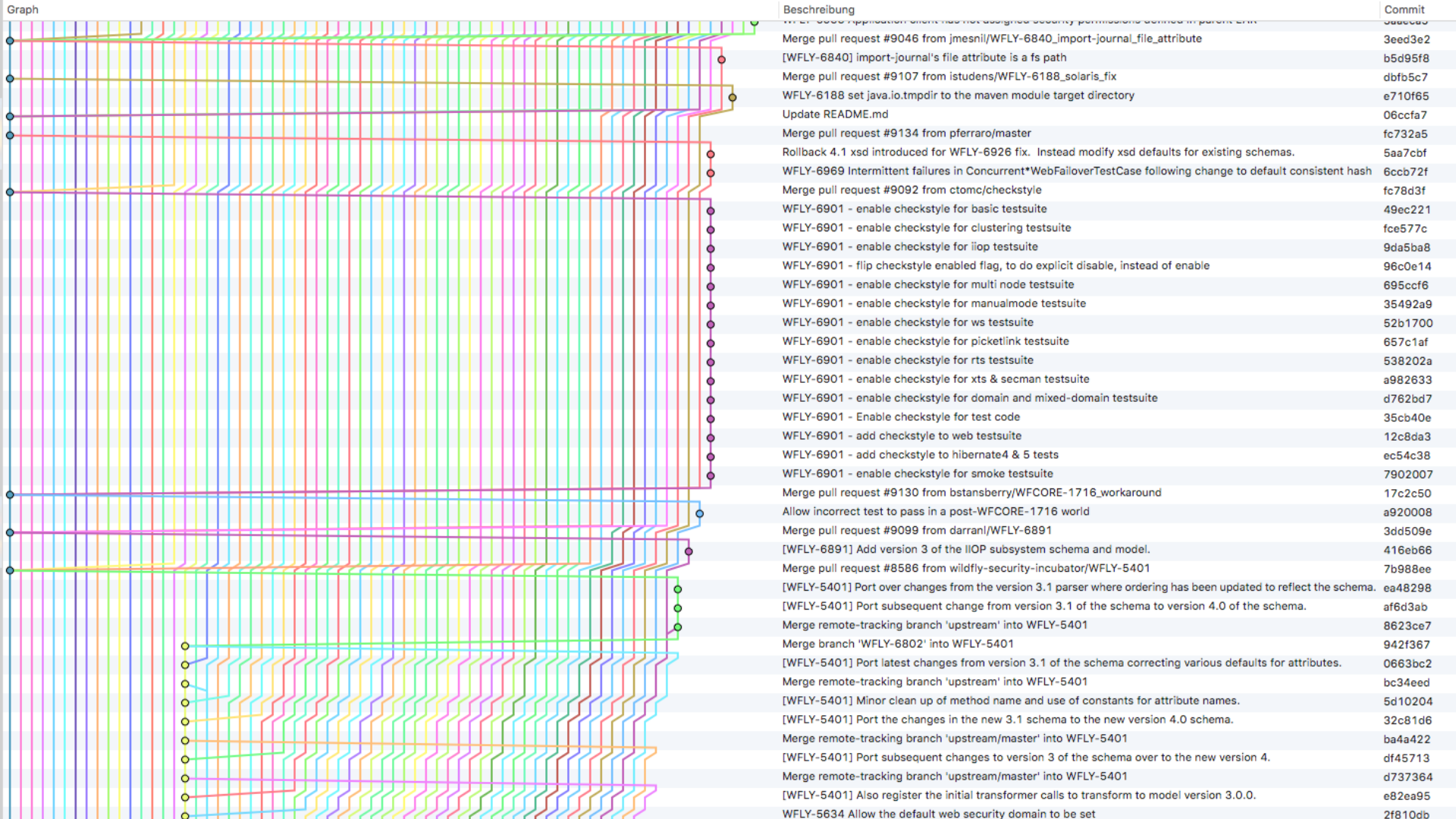
- Änderungen kommen sehr oft vor
- Teams und Prozesse müssen entsprechend reagieren
- Adequate Behandlung von Änderungswünschen ist essentiell
- Bereits vorher (rechtzeitig!) Gedanken machen!



VERSIONSMANAGEMENT

Motivation

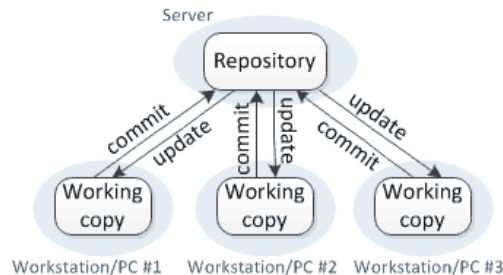
- Teams sind oft sehr groß
- Fehler können passieren und man möchte lokale Änderungen rückgängig machen
- Unterstützt das Verständnis der Abläufe während der Entwicklung
- Unterstützt die Zusammenarbeit von Entwicklern und Teams
- Entwicklung kann sehr schnell, sehr komplex werden



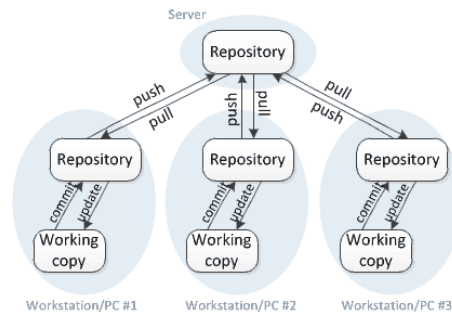
Versionsmanagement – Terminologie

- Die Versionsverwaltung kann wie folgt erfolgen:
 - **Zentral:** Client/Server System, wobei es nur einen Server gibt. **Beispiele:** CVS, SVN
 - **Dezentral:** Ein Repository pro Client, jeder arbeitet auf seinem eigenen Entwicklungszweig. **Beispiel:** Git
 - **Lokal:** Nur für die lokale Entwicklung/Versionierung bestimmt. Viele der genannten Konfigurationssysteme können auch lokal verwendet werden

Centralized version control



Distributed version control



Source Code Management – GIT

- Git
 - Tool für verteiltes Versionsmanagement
 - Frei verfügbar
 - Konzepte auch bei anderen Tools sehr ähnlich
- Ein üblicher Ablauf in Git beinhaltet die folgenden Aktionen
 - Repository vom Server downloaden (clone)
 - Änderungen lokal speichern (commit)
 - Lokale Änderungen an den Server schicken (push)
 - Änderungen vom Server abholen (pull)
- Zusätzlich können auch Verzweigungen und Markierungen erstellt werden
 - Beginnen eines neuen Entwicklungsstrangs (branch)
 - Markieren einer Revision (tag)
- Fortgeschrittene Techniken
 - rebase, stash, checkout, diff,...



Source Code Management – GIT

- Clone
 - Kopiert das komplette Repository von einem Server auf den lokalen Rechner
 - Danach können lokal Änderungen durchgeführt werden
- Commit
 - Gibt die Änderungen „frei“ und speichert sie in einer neuen Revision
 - Revisionen in Git werden mittels SHA-1 Hashes identifiziert (z.B.: 2e3413e918deaba1afb08038d0de609204e141ad)



- **Jede** Änderung ist kurz zu kommentieren (was und warum) → Commit Message

Source Code Management – GIT

- push
 - Überträgt die Änderungen die mittels commit freigegeben wurden zum Server
 - Lokales Repository muss „sauber“ sein (keine Änderungen von anderen Entwicklern am Server), sonst muss zuerst ein „pull“ erfolgen
- pull
 - Holt die Änderungen der anderen Entwickler vom Server ab und versucht sie in den lokalen Code Stand zu integrieren (merge)
 - Manche Änderungen können nicht automatisch zusammen geführt werden
→ Konflikt

Source Code Management – GIT

- branch
 - Erstellt eine neue Verzweigung
 - Sehr nützlich zur Vermeidung von Merge-Konflikten
 - Änderungen werden nach Fertigstellung in den Hauptstrang integriert
- tag
 - Markiert eine Revision
 - Wird z.B. verwendet um Releases zu markieren (z.B. „4.2.3“)

Wann ist eine Commit Message gut?



	COMMENT	DATE
○	CREATED MAIN LOOP & TIMING CONTROL	14 HOURS AGO
○	ENABLED CONFIG FILE PARSING	9 HOURS AGO
○	MISC BUGFIXES	5 HOURS AGO
○	CODE ADDITIONS/EDITS	4 HOURS AGO
○	MORE CODE	4 HOURS AGO
○	HERE HAVE CODE	4 HOURS AGO
○	AAAAAAA	3 HOURS AGO
○	ADKFJSLKDFJSDKLFJ	3 HOURS AGO
○	MY HANDS ARE TYPING WORDS	2 HOURS AGO
○	HAAAAAAAAAANDS	2 HOURS AGO

AS A PROJECT DRAGS ON, MY GIT COMMIT
MESSAGES GET LESS AND LESS INFORMATIVE.

(Some) Rules of a Great Commit Message

1. Zusammenfassung und Details trennen
2. Zusammenfassung auf 50 Zeichen begrenzen
3. Referenzen angeben (z.B. Links zum Issue Tracker)

(Some) Rules of a Great Commit Message

1. Zusammenfassung und Details trennen
2. Zusammenfassung auf 50 Zeichen begrenzen
3. Referenzen angeben (z.B. Links zum Issue Tracker)

```
$ git log
commit 42e769bdf4894310333942ffc5a15151222a87be
Author: Kevin Flynn <kevin@flynnsarcade.com>
Date:   Fri Jan 01 00:00:00 1982 -0200

Derezz the master control program

MCP turned out to be evil and had become intent on world domination.
This commit throws Tron's disc into MCP (causing its deresolution)
and turns it back into a chess game.
```

```
$ git log --oneline
42e769 Derezz the master control program
```

- Message Body oft nicht sichtbar
- Trennung von Zusammenfassung und Details

(Some) Rules of a Great Commit Message

1. Zusammenfassung und Details trennen
2. Zusammenfassung auf 50 Zeichen begrenzen
3. Referenzen angeben (z.B. Links zum Issue Tracker)



Commit changes

 **ProTip:** Great commit summaries are 50 characters or less. Place extra information in the extended description.

Demonstrate a subject line that goes on too long and gets truncated by the GitHub UI

Add an optional extended description...

Commits on Aug 31, 2014



Demonstrate a subject line that goes on too long and gets truncated b...

cbeams authored 2 minutes ago

- GitHub trennt nach 72 Zeichen ab (50 anpeilen)
- Zusammenfassung kurz halten

(Some) Rules of a Great Commit Message

1. Zusammenfassung und Details trennen
2. Zusammenfassung auf 50 Zeichen begrenzen
3. Referenzen angeben (z.B. Links zum Issue Tracker)

```
$ git log --oneline  
42e769 [MYISSUE-1234] Derezz the master control program
```

VS.

```
$ git log --oneline  
42e769 Derezz the master control program
```

- Details können beim Issue nachgelesen werden
- Zuordnung/Aggregation leicht möglich

Beispiele



The screenshot shows a code editor window for the file `wicket-util/src/test/java/org/apache/wicket/util/convert/converters/ConvertersTest.java`. The editor displays a diff of Java code. The left margin shows line numbers, and the right margin shows the corresponding line numbers in the original file. The code is as follows:

```
@@ -24,6 +24,7 @@
24 24 import java.math.BigDecimal;
25 25 import java.text.ChoiceFormat;
26 26 import java.text.NumberFormat;
27 + import java.text.ParsePosition;
27 28 import java.util.Calendar;
28 29 import java.util.Date;
29 30 import java.util.Locale;

@@ -49,7 +50,7 @@
49 50 private static final Locale DUTCH_LOCALE = new Locale("nl", "NL");
50 51
51 52 @Test
52 - void thousandSeperator() {
53 + void thousandSeparator() {
53 54     BigDecimalConverter bdc = new BigDecimalConverter();
54 55     assertEquals(new BigDecimal(3000), bdc.convertToObject("3 000", Locale.FRENCH));
55 56
```

The diff highlights show the following changes:

- Line 27: Added `import java.text.ParsePosition;` (green highlight).
- Line 52: Changed `thousandSeperator` to `thousandSeparator` (green highlight).

Beispiele



```
Expand All  wicket-util/src/test/java/org/apache/wicket/util/convert/converters/ConvertersTest.java ...  
@@ -24,6 +24,7 @@  
24 24 import java.math.BigDecimal;  
25 25 import java.text.ChoiceFormat;  
26 26 import java.text.NumberFormat;  
27 + import java.text.ParsePosition;  
27 28 import java.util.Calendar;  
28 29 import java.util.Date;  
29 30 import java.util.Locale;  
@@ -49,7 +50,7 @@  
49 50 private static final Locale DUTCH_LOCALE = new Locale("nl", "NL");  
50 51  
51 52 @Test  
52 - void thousandSeperator() {  
53 + void thousandSeparator() {  
53 54     BigDecimalConverter bdc = new BigDecimalConverter();  
54 55     assertEquals(new BigDecimal(3000), bdc.convertToObject("3 000", Locale.FRENCH));  
55 56
```

„Fix typos in test method names.“

Beispiele

pom.xml			...
Lin.	Col.	Content	
221	221	<version>1.0.10</version>	
222	222	<type>jar</type>	
223	223	</dependency>	
224	-	<dependency>	
225	-	<groupId>com.google.guava</groupId>	
226	-	<artifactId>guava</artifactId>	
227	-	<version>27.0.1-jre</version>	
228	-	</dependency>	
229	224	<dependency>	
230	225	<groupId>com.google.inject</groupId>	
231	226	<artifactId>guice</artifactId>	
232			

wicket-core/pom.xml			...
Lin.	Col.	Content	
67	67	<groupId>org.danekja</groupId>	
68	68	<artifactId>jdk-serializable-functional</artifactId>	
69	69	</dependency>	
70	-	<dependency>	
71	-	<groupId>com.google.guava</groupId>	
72	-	<artifactId>guava</artifactId>	
73	-	<scope>test</scope>	
74	-	</dependency>	
75	70	<dependency>	
76	71	<groupId>org.apache.commons</groupId>	
77	72	<artifactId>commons-lang3</artifactId>	
78			

Beispiele

„Remove
dependency on
Guava

Stopwatch class was
used by a single
test“

pom.xml		
Line	Start	End
221	221	<version>1.0.10</version>
222	222	<type>jar</type>
223	223	</dependency>
224	-	<dependency>
225	-	<groupId>com.google.guava</groupId>
226	-	<artifactId>guava</artifactId>
227	-	<version>27.0.1-jre</version>
228	-	</dependency>
229	224	<dependency>
230	225	<groupId>com.google.inject</groupId>
231	226	<artifactId>guice</artifactId>

wicket-core/pom.xml		
Line	Start	End
67	67	<groupId>org.danekja</groupId>
68	68	<artifactId>jdk-serializable-functional</artifactId>
69	69	</dependency>
70	-	<dependency>
71	-	<groupId>com.google.guava</groupId>
72	-	<artifactId>guava</artifactId>
73	-	<scope>test</scope>
74	-	</dependency>
75	70	<dependency>
76	71	<groupId>org.apache.commons</groupId>
77	72	<artifactId>commons-lang3</artifactId>

Beispiele – Tangled Commits

- Ein Commit beinhaltet die Änderungen mehrerer Tasks
→ Tangled (“Verknäult”) Commit

Beispiele – Tangled Commits

- Ein Commit beinhaltet die Änderungen mehrerer Tasks
→ Tangled (“Verknäult”) Commit
- Eher schlecht
 - Entwicklung vermeintlich einfacher
 - Retrospektive Analyse schwer
 - Finden von Code (z.B. zum Fehler beheben) schwer

Beispiele – Tangled Commits

- Ein Commit beinhaltet die Änderungen mehrerer Tasks
→ Tangled (“Verknäult”) Commit
- Eher schlecht
 - Entwicklung vermeintlich einfacher
 - Retrospektive Analyse schwer
 - Finden von Code (z.B. zum Fehler beheben) schwer

→ Vermeiden!

Beispiele – Tangled Commits

- Best Practice
 - Für jeden Task/Schritt ein Commit
 - Sinnvolle Message! (Was wurde in dem Schritt gemacht?)
 - Referenz zum Issue (z.B. [ISSUE-1234] <Message>)

Beispiele – (Un)Tangled Commits



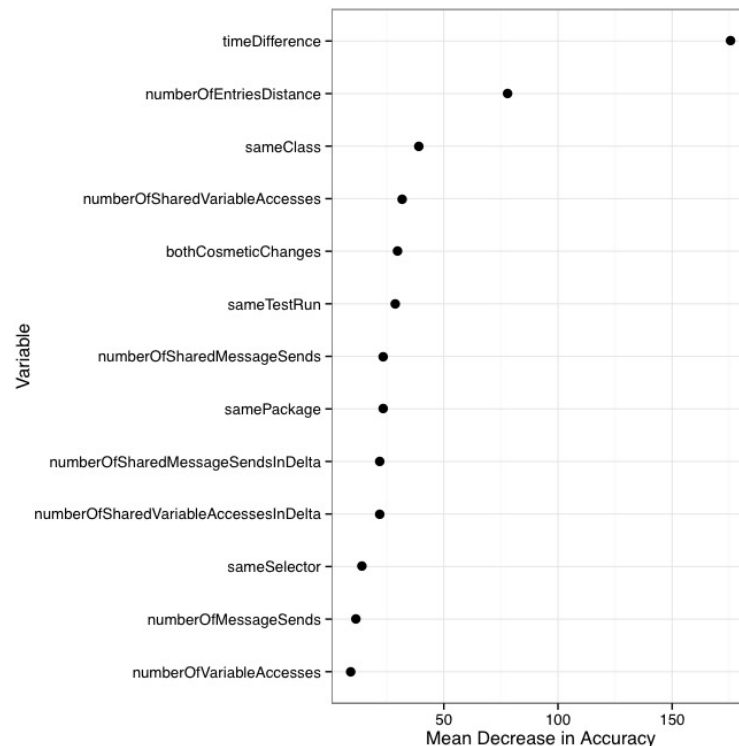
[WFLY-11930] Community documentation about which file descriptors (e.g jboss-permissions.xml, permissio...	Yeray Borges <y...	136d4fc8de	15.01.2020, 09:38
[WFLY-11930] Allow Java EE Subsystem to enable/disable descriptor based property replacement for permis...	Yeray Borges <y...	705f3535b0	15.01.2020, 09:38

Research?

„When developers commit software changes to a version control system, **they often commit unrelated changes** in a single transaction—simply because, while, say, fixing a bug in module A, **they also came across a typo in module B, and updated a deprecated call in module C.**“, Herzig and Zeller, 2011

Tool Support? Research?

- Beeinflussende Faktoren
 - Zeitliche Nähe
 - "Örtliche Nähe"
 - Gleiche Klasse
 - Gemeinsame Zugriffe auf Variablen
- (Etwas) Tool Support



Tool Support? Research?

Untangling Fine-Grained Code Changes

Martín Dias¹, Alberto Bacchelli², Georgios Gousios³, Damien Cassou¹, Stéphane Ducasse¹

1: RMoD Inria Lille–Nord Europe, University of Lille — CRISTAL, France

2: SORCERERS @ Software Engineering Research Group, Delft University of Technology, The Netherlands

3: Digital Security Group, Radboud Universiteit Nijmegen, The Netherlands

Abstract—After working for some time, developers commit their code changes to a version control system. When doing so, they often bundle unrelated changes (e.g., bug fix and refactoring) in a single commit, thus creating a so-called tangled commit. Sharing tangled commits is problematic because it makes review, reversion, and integration of these commits harder and historical analyses of the project less reliable.

Researchers have worked at untangling existing commits, i.e., finding which part of a commit relates to which task. In this paper, we contribute to this line of work in two ways: (1) A publicly available dataset of untangled code changes, created with the help of two developers who accurately split their code changes into self contained tasks over a period of four months; (2) a novel approach, *EpiceaUntangler*, to help developers share untangled commits (aka. atomic commits) by using fine-grained code change information. *EpiceaUntangler* is based and tested on the publicly available dataset, and further evaluated by deploying it to 7 developers, who used it for 2 weeks. We recorded a median success rate of 91% and average one of 75%, in automatically creating clusters of untangled fine-grained code changes.

I. INTRODUCTION

Version Control Systems (VCS), such as Git and Subversion, allow programmers to control changes to source code and make

to its own line). Sharing tangled commits is problematic as they make code review, reversion, and integration harder and historical analyses of the project less reliable [4]. For example, even integrating the code formatting change included in the aforementioned commit (without the refactoring) would be a demanding task.

Untangling existing commits (i.e., finding how to separate parts of a commit relating to different tasks) is an open research problem. Herzig and Zeller presented the earliest and most significant results in this area [3]: They implemented the first algorithm that can automatically untangle commits given artificially tangled ones.

In this paper, we expand on this previous work by: (1) working in an untyped setting where a part of the approach by Herzig and Zeller is inapplicable; (2) considering fine-grained code change information gathered during development (e.g., time at which each line has changed and all versions of each line); and (3) evaluating the resulting approach both on data generated by programmers who manually label it and with programmers working on real-world development tasks.

The ultimate goal of our work is to help developers of

mean Decrease in Accuracy

- Beinflusst
- Zeitlich
- "Örtlich
- Gleich
- Gemein
- auf Vari
- (Etwas)

Source Code Management – Merging

- Was passiert wenn 2 Entwickler getrennt von einander, parallel etwas bearbeiten?

Source Code Management – Merging

- Was passiert wenn 2 Entwickler getrennt von einander, parallel etwas bearbeiten?
- Verschiedene Stellen → Kein Problem
- Gleiche Stellen → Evtl. problematisch

Source Code Management – Merging

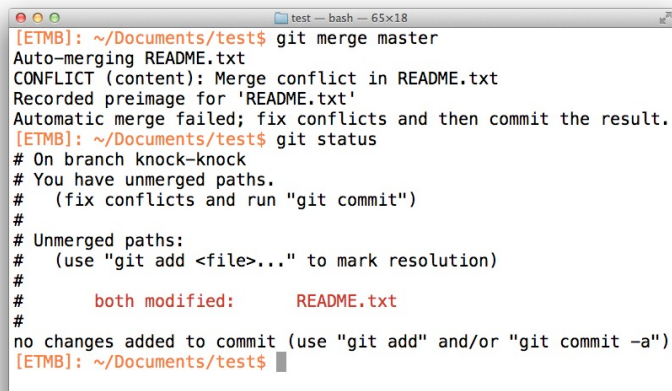
- Gleichzeitige Änderung
in der Datei `Main.java`

→ Zusammenführen
notwendig! (=Merging)



Source Code Management – Merging

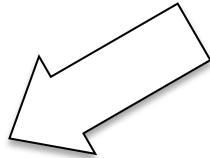
- Soweit möglich von Git übernommen
- Sonst:

A terminal window titled 'test -- bash -- 65x18' showing the output of a failed git merge. The user runs 'git merge master', which fails due to a conflict in 'README.txt'. The terminal shows the conflict message, the preimage for 'README.txt', and a message that the automatic merge failed. The user then runs 'git status', which shows that 'README.txt' is both modified and unmerged. The terminal text is as follows:

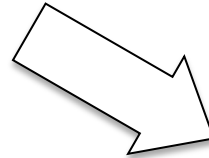
```
[ETMB]: ~/Documents/test$ git merge master
Auto-merging README.txt
CONFLICT (content): Merge conflict in README.txt
Recorded preimage for 'README.txt'
Automatic merge failed; fix conflicts and then commit the result.
[ETMB]: ~/Documents/test$ git status
# On branch knock-knock
# You have unmerged paths.
#   (fix conflicts and run "git commit")
#
# Unmerged paths:
#   (use "git add <file>..." to mark resolution)
#
#       both modified:   README.txt
#
no changes added to commit (use "git add" and/or "git commit -a")
[ETMB]: ~/Documents/test$
```

Source Code Management – Merging

Beim Zusammenführen von Änderungen kann es, ohne geeignete Strategien, zu Problemen kommen



Überschreiben von Änderungen
anderer Entwickler



OUTDATED

Arbeiten an nicht mehr aktuellen
Codeständen

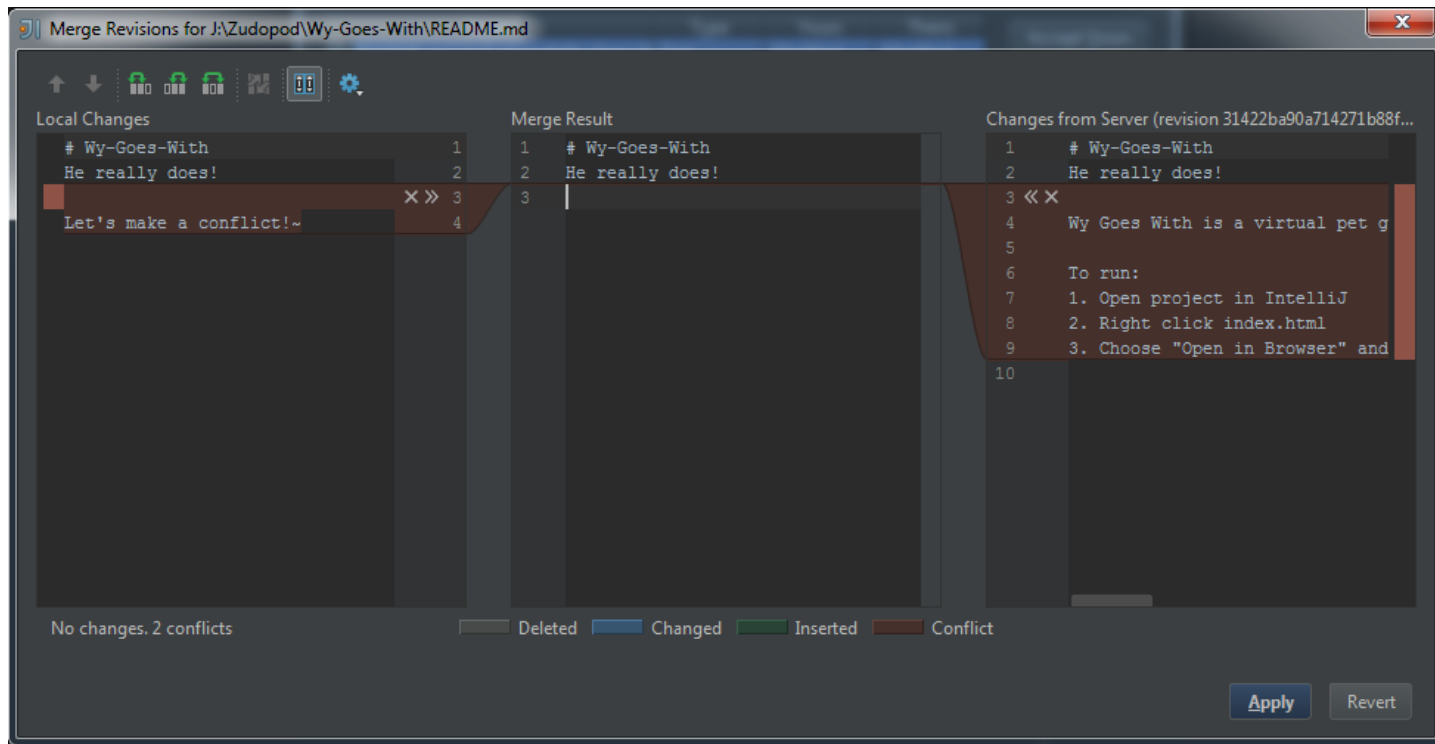
Source Code Management – Merging

- Lösungsstrategien
 - **Lock-Modify-Unlock:** Beim Auschecken der Dateien werden diese für andere gesperrt. Der Code kann modifiziert werden und erst nach einem Commit steht dieser wieder anderen Entwicklern zur Änderung zur Verfügung.
 - Nachteil: Lange Wartezeiten wenn die Komponente gesperrt ist und der Entwickler der die Komponente sperre nicht erreichbar ist
 - **Copy-Modify-Merge:** Auschecken ist jederzeit erlaubt. Beim Einchecken entsteht mitunter aber ein Konflikt, welcher durch Zusammenfügen (Merge) der Codestellen gelöst werden muss.
 - Nachteil: das Zusammenfügen ist mitunter nicht immer automatisch möglich
 - **Feature Branches:** Entwickler arbeitet in einem eigenen Branch und integriert die Änderungen am Ende der Arbeit

Source Code Management – Merging

- Lösungsstrategien
 - **Lock-Modify-Unlock:** Beim Auschecken der Dateien werden diese für andere gesperrt. Der Code kann modifiziert werden und erst nach einem Commit steht dieser wieder anderen Entwicklern zur Änderung zur Verfügung.
 - Nachteil: Lange Wartezeiten wenn die Komponente gesperrt ist und der Entwickler der die Komponente sperre nicht erreichbar ist
 - **Copy-Modify-Merge:** Auschecken ist jederzeit erlaubt. Beim Einchecken entsteht mitunter aber ein Konflikt, welcher durch Zusammenfügen (Merge) der Codestellen gelöst werden muss.
 - Nachteil: das Zusammenfügen ist mitunter nicht immer automatisch möglich
 - **Feature Branches:** Entwickler arbeitet in einem eigenen Branch und integriert die Änderungen am Ende der Arbeit

Source Code Management – Merging



Source Code Management – Merging

- <<<<<<, ===== und >>>>>> zeigen Bereiche von Konflikten an
 - Beheben durch entfernen der Marker und manuellem Zusammenfügen des Codes (merge)

```
# Wy-Goes-With
He really does!

<<<<<< HEAD
Let's make a conflict!~
=====
Wy Goes With is a virtual pet game that runs in the browser using phaser.io and Spine. This project is just for fun. :)

To run:
1. Open project in IntelliJ
2. Right click index.html
3. Choose "Open in Browser" and choose a browser
>>>>>> 31422ba90a714271b88f4ba0b8f505a93e41fda5
```


Source Code Management – Merging

- <<<<<<, ===== und >>>>>> zeigen Bereiche von Konflikten an
 - Beheben durch entfernen der Marker und manuellem Zusammenfügen des Codes (merge)

```
# Wy-Goes-With  
He really does!
```

„Original“

```
<<<<<< HEAD  
Let's make a conflict!~  
=====
```

Version 1

```
Wy Goes With is a virtual pet game that runs in the browser using phaser.io and Spine. This project is just for fun. :)  
  
To run:  
1. Open project in IntelliJ  
2. Right click index.html  
3. Choose "Open in Browser" and choose a browser  
>>>>>> 31422ba90a714271b88f4ba0b8f505a93e41fda5
```

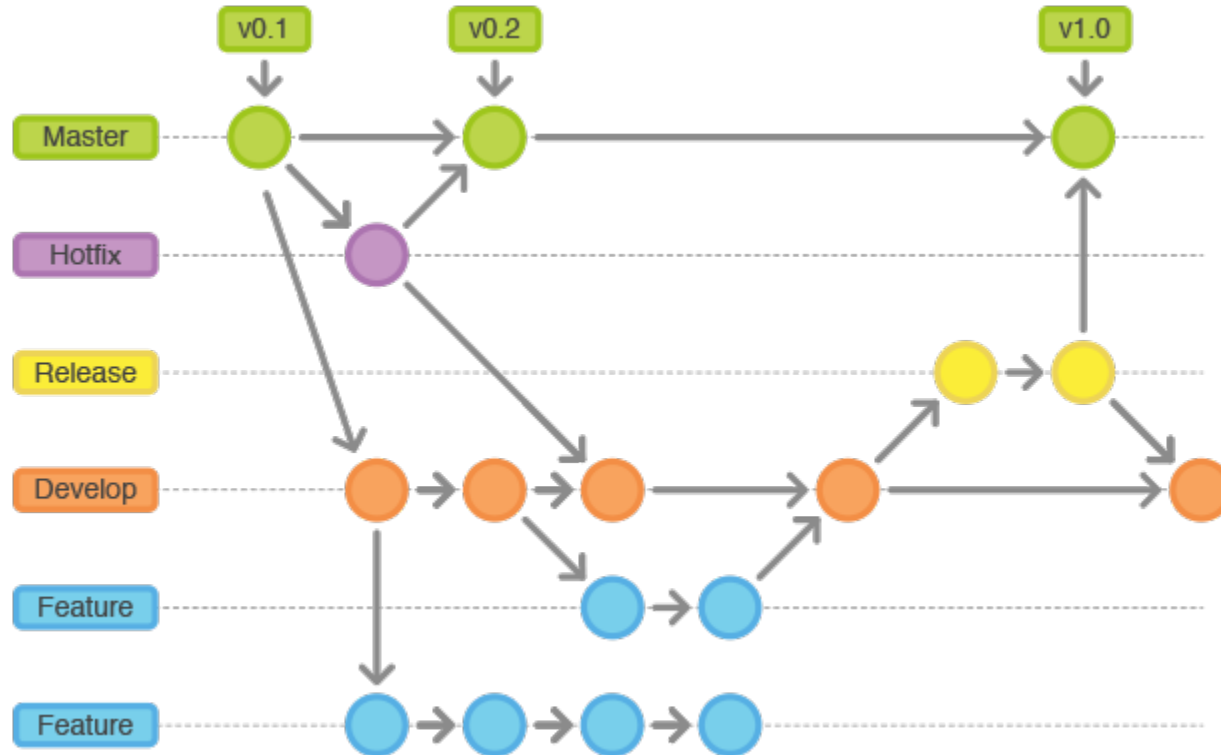
Version 2

Source Code Management – GIT

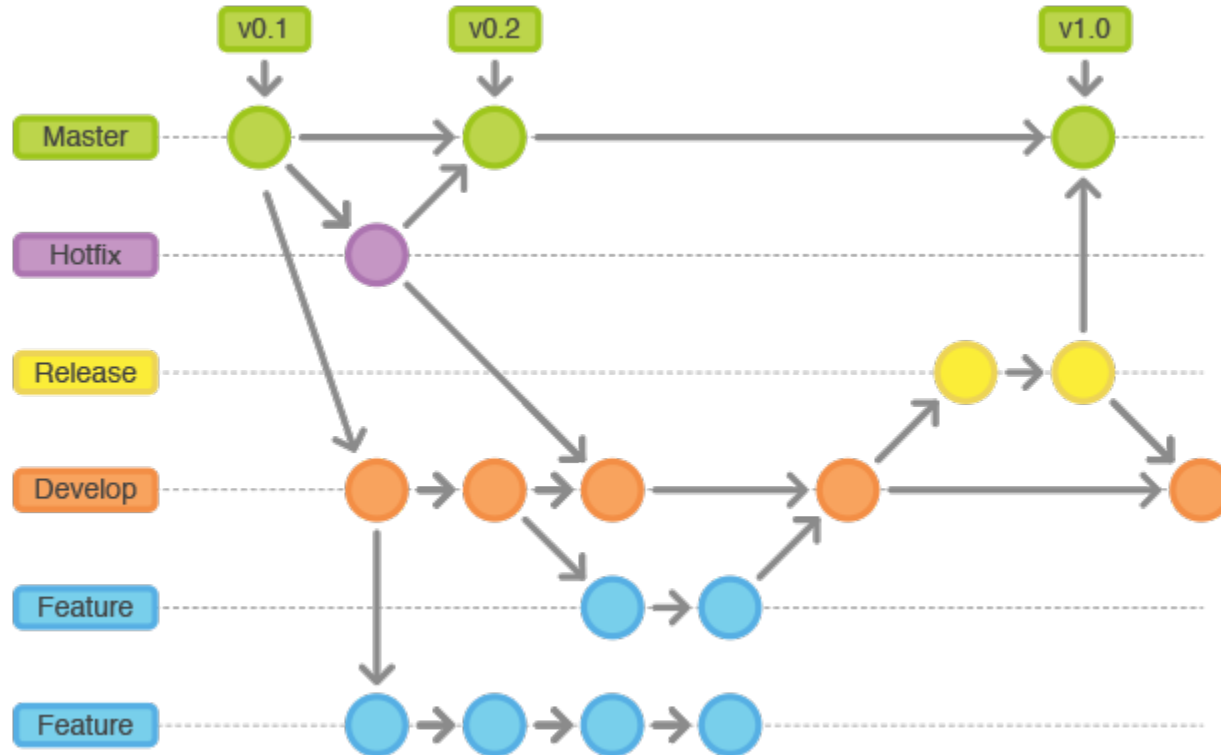
- Es hat sich folgendes Modell als sehr nützlich erwiesen, um im Team zu entwickeln (auch für die SE2 Übung ;))



Source Code Management – GIT

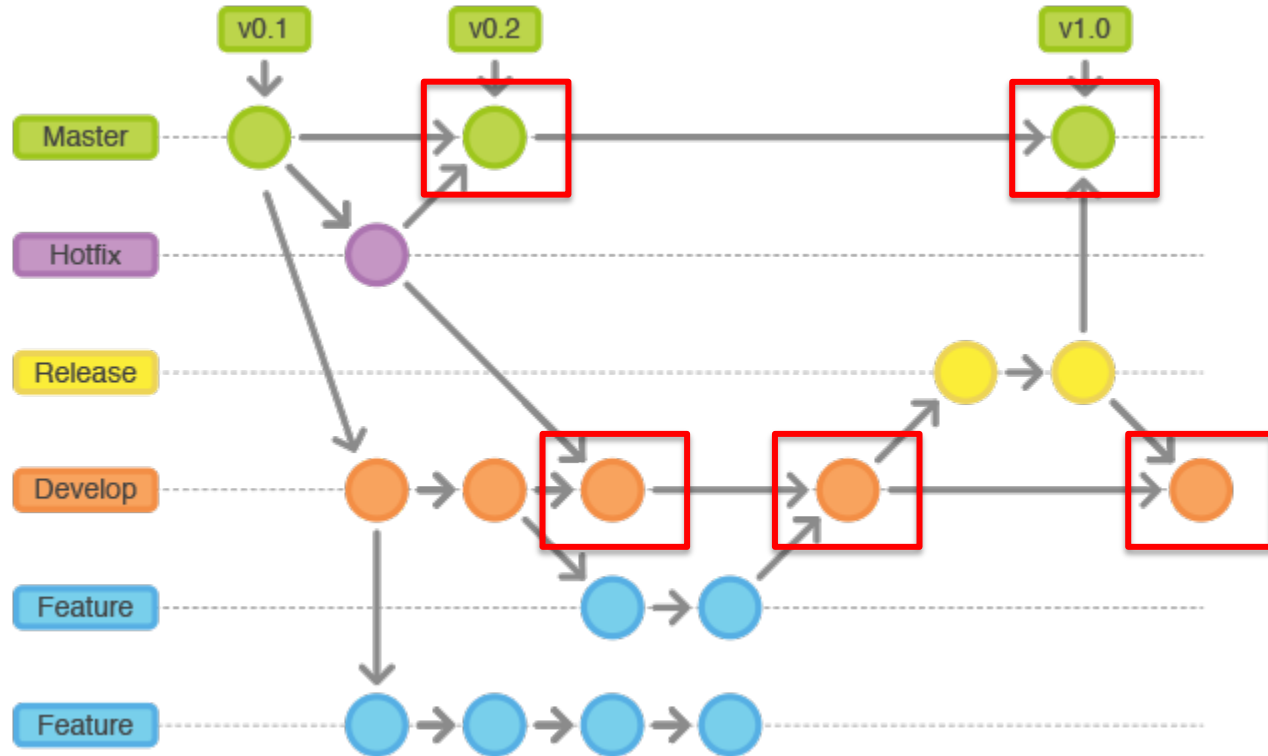


Source Code Management – GIT



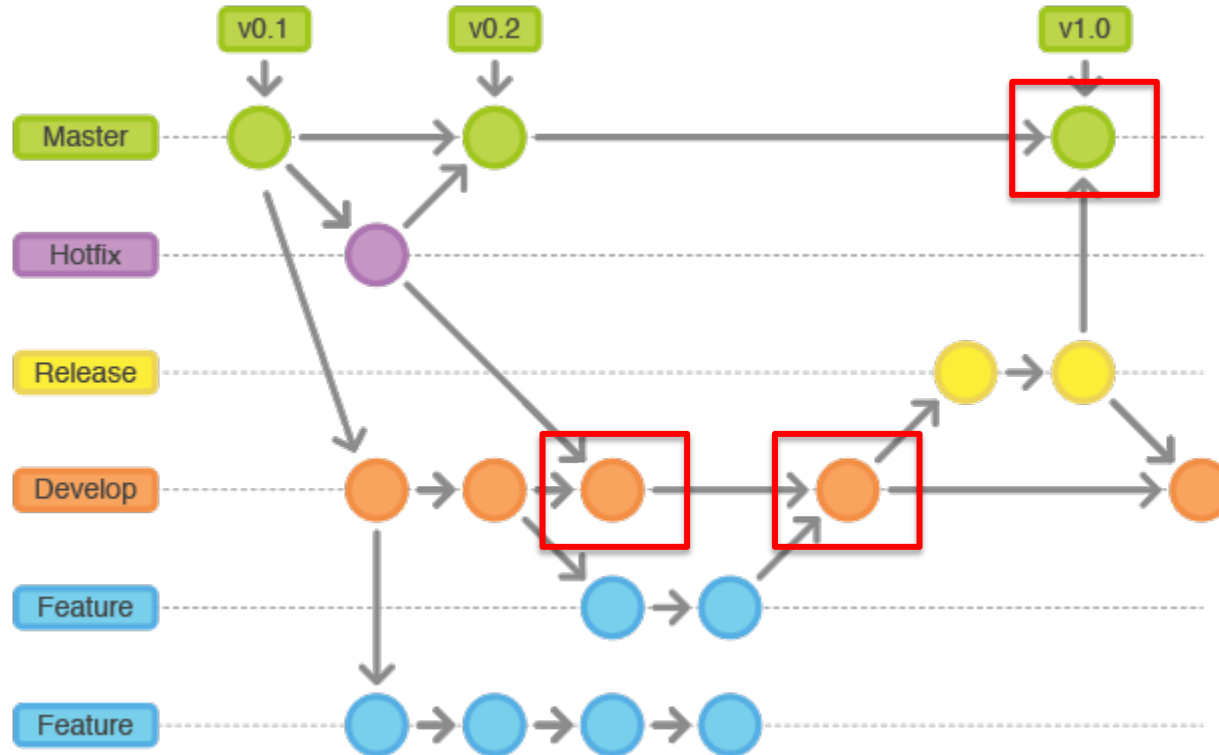
Wo können
Merge
Conflicts
auftreten?

Source Code Management – GIT



Wo können
Merge
Conflicts
auftreten?

Source Code Management – GIT



Wo können
Merge
Conflicts
auftreten?

Semantic Versioning – Muster

MAJOR.MINOR.PATCH

- Version Zahlen verwenden üblicherweise dieses Muster (e.g. 1.3.15)
- MAJOR: erhöhen bei **inkompatiblen API Änderungen**
- MINOR: hinzufügen von Funktionalität in **einer abwärtskompatiblen Art**
- PATCH: kleinere Änderungen, wie z.B. abwärtskompatible **Bug Fixes**

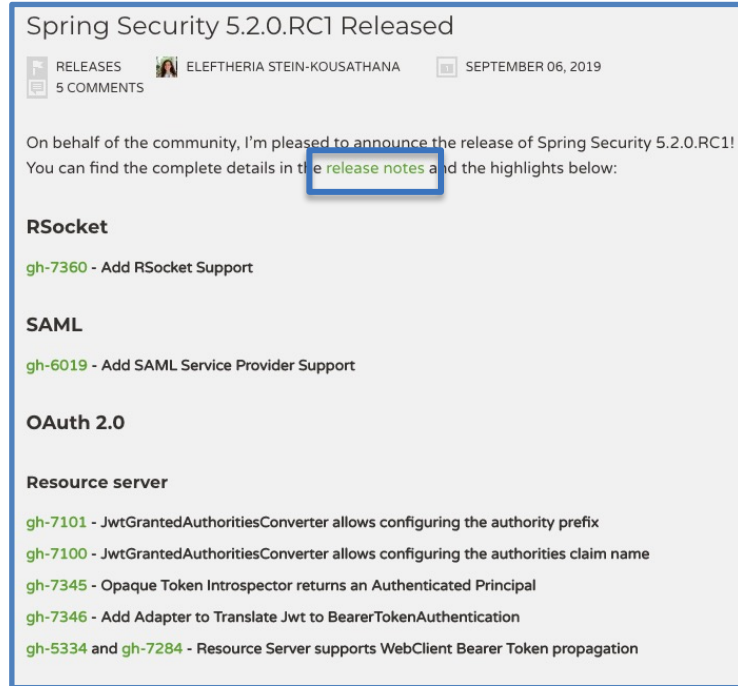
Source: <https://semver.org/>

Semantic Versioning – Postfixes

- Manchmal werden auch noch folgende Endungen verwendet:
 - SNAPSHOT: während der Entwicklung, „under frequent change“ (*1.2.1-SNAPSHOT*)
 - M<Zahl>: Zeigt Milestone Releases an (*2.4.1-M1, 2.4.1-M2, ...*)
 - Beta: Zeigt Beta Testversionen an (*1.3.1-b1*)
 - RC oder CR: Release Candidate, quasi finale Version die ausgebracht werden soll (*1.3.0-rc4*)
 - GA: “General Availability”, Öffentlicher Release (*1.0.4.GA*)
 - Release: finale Version

Release Notes

- Inhalte variieren
- Häufig folgende Inhalte
 - Versionsnummer
 - Datum (des Releases)
 - Zusammenfassung
 - Fixed bugs
 - Neue Features
 - Abhängigkeitsänderungen
 - Referenzen zu Issues (gh-xxx)
 - Mitarbeiter (Contributors)
 - Download Links/Dependency Definitions (e.g. für Maven/Gradle)



Release Notes – Beispiel

Pre-release

5.2.0.RC1

ecf0062

Verified

5.2.0.RC1

 jzheaux released this 19 days ago · [66 commits](#) to master since this release

★ New Features

- Add attributes Consumer to OAuth2AuthorizationContext [#7385](#)
- Improve DefaultReactiveOAuth2UserService handling IOException [#7370](#)
- Add RSocket Support [#7360](#)
- Polish Server|ServletBearerExchangeFilterFunction [#7355](#)
- Refactor Servlet/Server BearerExchangeFilterFunction [#7353](#)

🐛 Bug Fixes

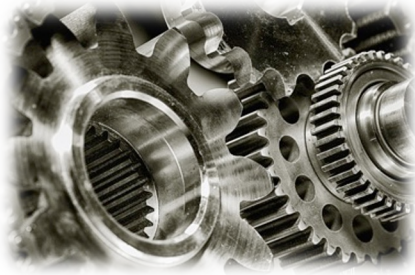
- Remove package tangle in headers [#7380](#)
- Remove OAuth2AuthorizationRequest when a distributed session is used [#7334](#)
- OAuth2AuthorizationRequest not removed from session [#7327](#)
- Use ConcurrentHashMap in InMemoryReactiveClientRegistrationRepository [#7308](#)
- fix footnotes markup [#7305](#)

🔨 Dependency Upgrades

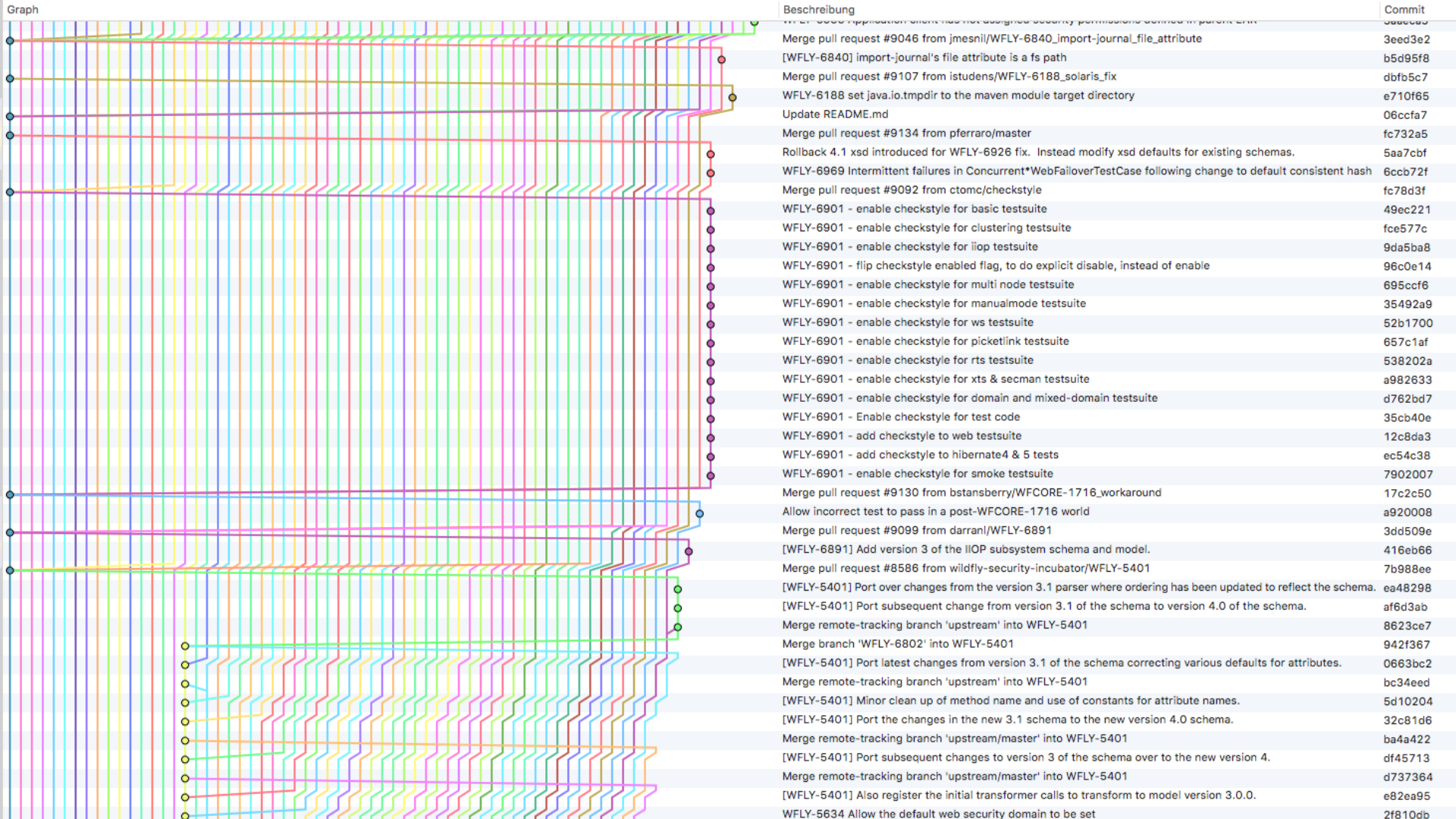
- Update to Gretty 2.3.1 [#7389](#)
- Update to OpenSaml 3.3.1 [#7388](#)
- Update to cglib 3.3.0 [#7387](#)

Zusammenfassung

- Zentralisiertes vs. Verteiltes Versionsmanagement
- Workflow für Source Code Management Systeme
 - clone
 - commit
 - push
 - pull
- Best Practice für die Nutzung von GIT
 - Feature Branches
 - Hotfix Branch
 - Release Branch
 - Master Branch
- Tagging von Versionen/Releases



BUILDMANAGEMENT/AUTOMATISIERUNG



Motivation

- Viele Branches und Entwicklungsstränge
 - Viele Versionen die kompiliert, gebaut, getestet, und ausgebracht werden müssen
 - Ohne Automatisierung nur für kleine Projekte möglich
 - Amazon: 136,000 builds/deployments pro Tag
- Software Automation Pipeline

Motivation

Microsoft Internal Engineering in VSTS by the Numbers



Data: Internal Microsoft engineering system activity during calendar year 2017

Motivation

Microsoft Internal Engineering in VSTS by the Numbers



Data: Internal Microsoft engineering system activity during calendar year 2017

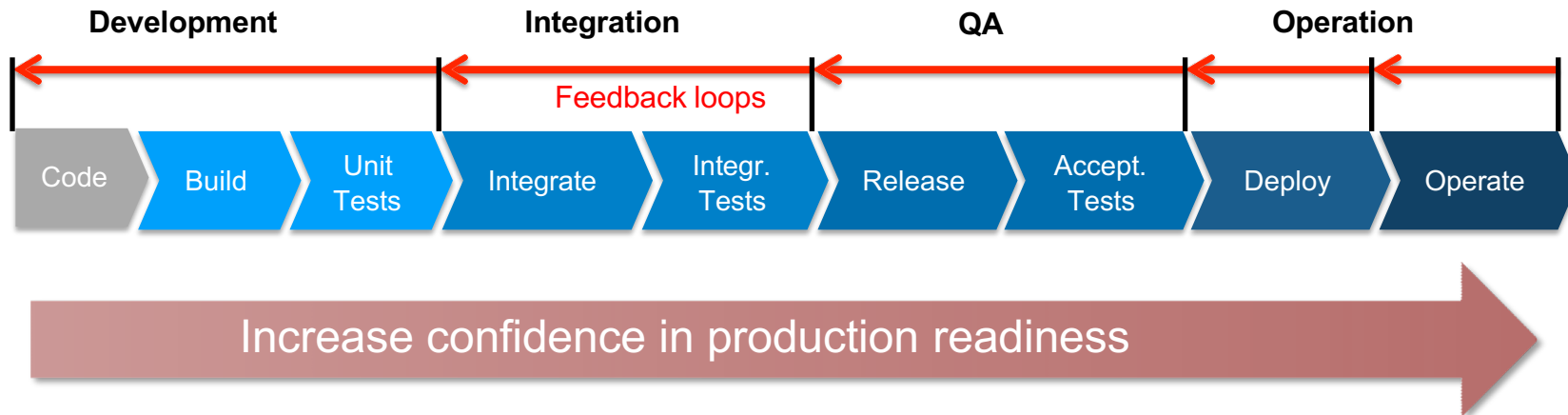
Manually???



VS.

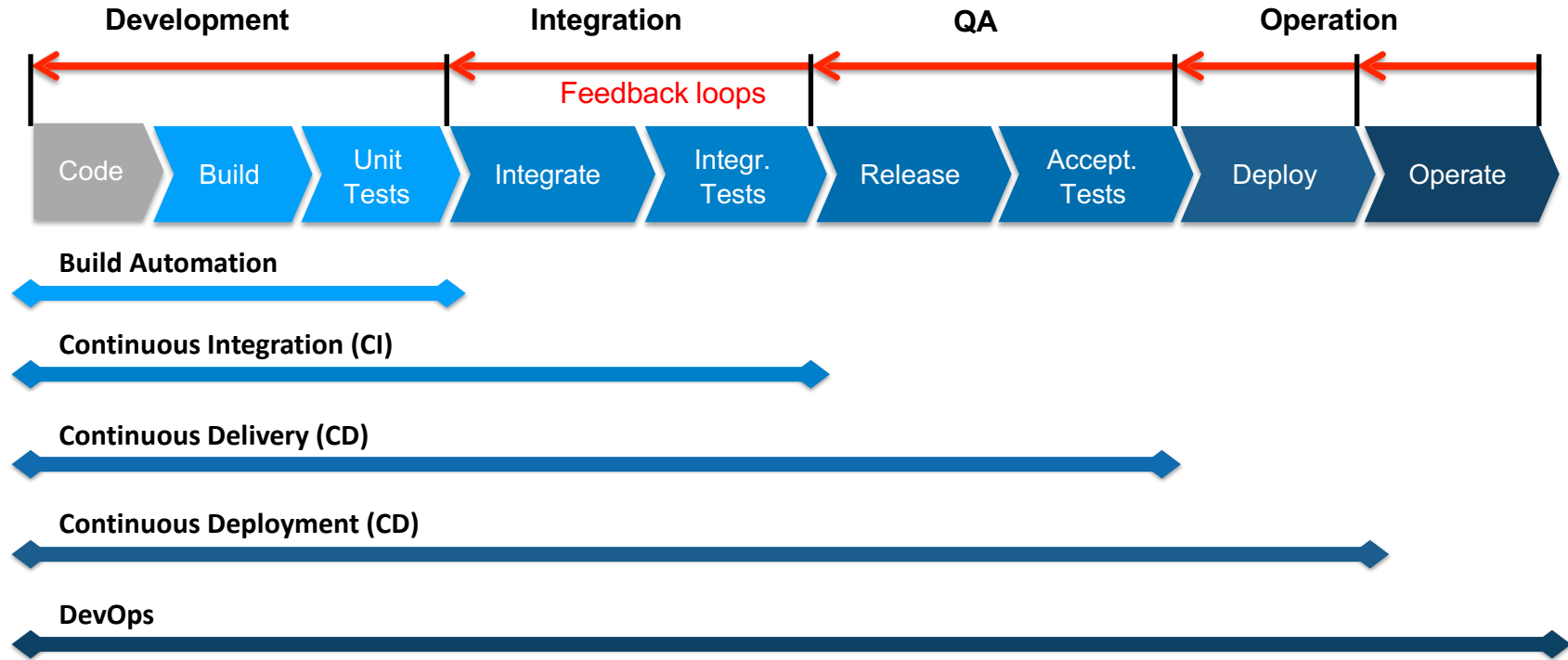


Software Automation Pipeline



- Die **Abläufe jeder Phase des Software Prozesses** können **automatisiert** werden (von Entwicklung zu Integration bis zu QA und Betrieb)
- Weitere **Schritte** werden **nur (automatisch) ausgeführt** wenn einige **Bedingungen erfüllt** worden sind
- **Andernfalls** werden die Verantwortlichen über den Fehler **benachrichtigt** und der Prozess angehalten → **Feedback loop**

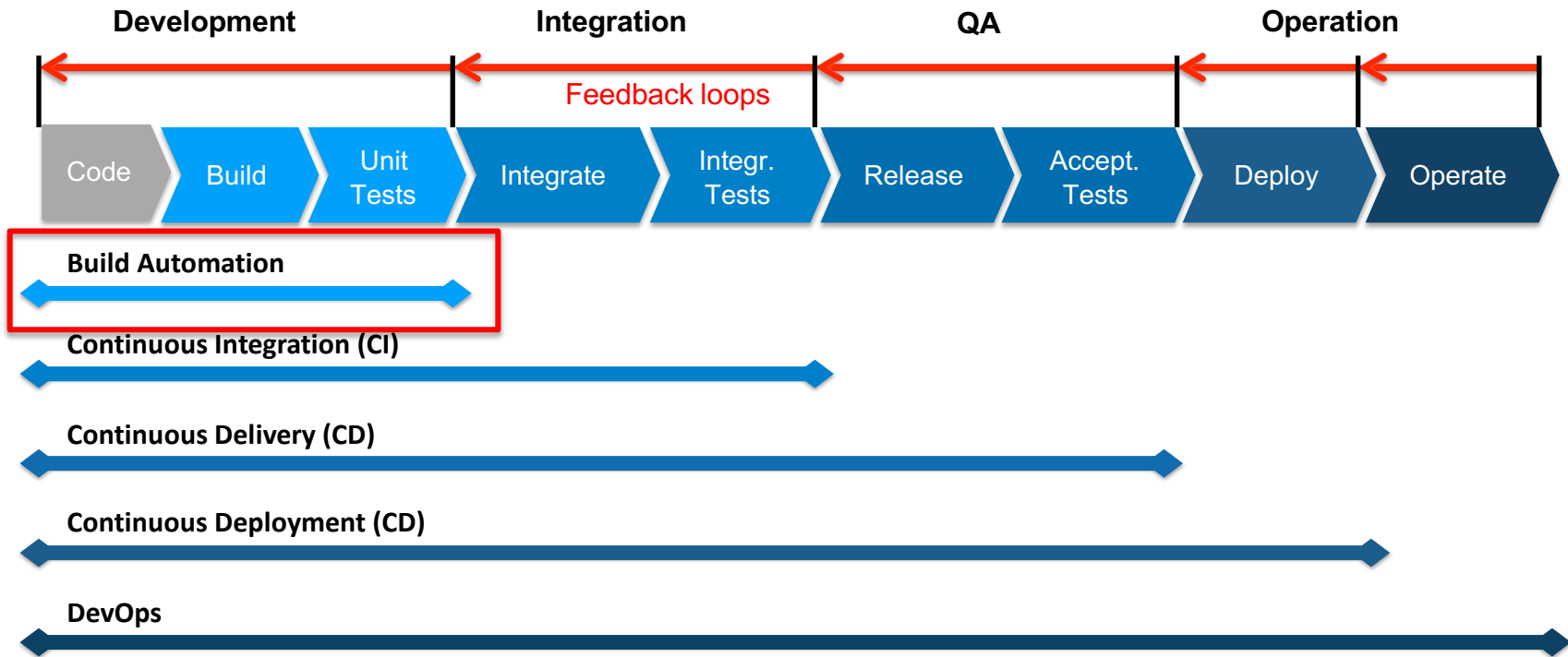
Software Automation Pipeline



Software Automation Pipeline

- Build Automation
 - Bauen von individuellen Komponenten und Ausführung von Unit Tests
 - Typischer Weise vom Entwickler ausgeführt
- Continuous Integration
 - Automatisches Bauen, Testen, und Integrieren von Komponenten und Ausführung von Integrations Tests
 - Typischer Weise am Continuous Integration Server ausgeführt
- Continuous Delivery
 - Zusätzliches Erstellen von Releases, deren Ausbringung auf Test Umgebungen, und Ausführen von Akzeptanztests
 - Release ist bereit um in die Produktivumgebung ausgebracht zu werden
- Continuous Deployment
 - Ausbringung des Releases auf die Produktivumgebung
- DevOps
 - Betrieb läuft automatisch ab (Backups, Monitoring, Scaling,...)

Software Automation Pipeline



Build Automation

- Build Automation deckt die Build Phase des SW lifecycle ab
- Wichtige Schritte sind
 - Auflösen von Abhängigkeiten (Dependency Resolution)
 - Kompilieren des Source Codes
 - Ausführung der (Unit) Tests
 - Software zusammenstellen (packaging)
 - Ausbringen in Repositories
 - Dokumentation erstellen
- Tools
 - Java: (Ant), Maven, Gradle,...
 - C/C++: make,...
 - Microsoft: MS Build, Visual Build,...
 - Ruby: Rake,...

maven



Ruby Rake



MSBuild

Build Automation/CI – Beispiel Maven

- Jedes Projekt besitzt ein Project Object Model (POM)
- XML (XSD-basierend)
- „Metadaten“ zum Projekt
 - Dependencies
 - Build Prozess
 - Dokumentation
 - ...
- POM kann vererbt werden (wie in OO)
 - „Super-POM“ wird von Maven bereit gestellt
 - Sinnvoll für große Projekte (z.B.: zentrale Verwaltung von Dependencies)

Build Automation/CI – Beispiel Maven

- (minimal) POM

```
1  <project xmlns="http://maven.apache.org/POM/4.0.0"
2      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
3      http://maven.apache.org/maven-v4_0_0.xsd"
4      <modelVersion>4.0.0</modelVersion>
5      <groupId>at.aau</groupId>
6      <artifactId>mavenexample</artifactId>
7      <version>1.0-SNAPSHOT</version>
8
9  </project>
10
```


Build Automation/CI – Beispiel Maven

- POM Bereiche (Abhängigkeiten)

```
31      <dependencies>
32          <dependency>
33              <groupId>junit</groupId>
34              <artifactId>junit</artifactId>
35              <version>3.8.1</version>
36              <scope>test</scope>
37          </dependency>
38          <dependency>
39              <groupId>commons-io</groupId>
40              <artifactId>commons-io</artifactId>
41              <version>2.5</version>
42          </dependency>
43      </dependencies>
```

Build Automation/CI – Beispiel Maven

- POM Bereiche (Build Prozess)

```
17  <build>
18    <plugins>
19      <plugin>
20        <groupId>org.apache.maven.plugins</groupId>
21        <artifactId>maven-release-plugin</artifactId>
22        <version>2.5.3</version>
23      </plugin>
24      <plugin>
25        <groupId>org.apache.maven.plugins</groupId>
26        <artifactId>maven-compiler-plugin</artifactId>
27        <version>3.7.0</version>
28        <configuration>
29          <source>1.8</source>
30          <target>1.8</target>
31        </configuration>
32      </plugin>
33    </plugins>
34  </build>
35
```

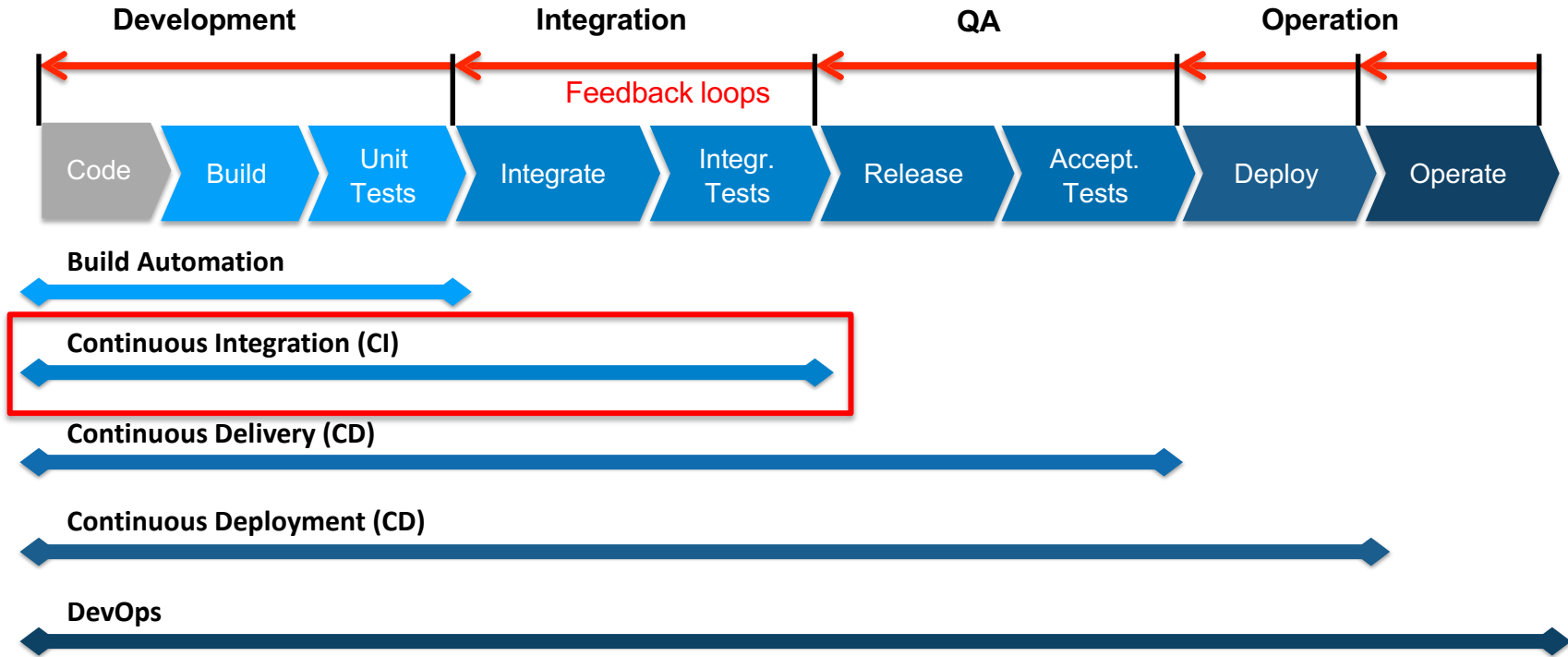
Build Automation/CI – Beispiel Maven

- Einfacher Aufruf
 - mvn [options] [<goal(s)>] [<phase(s)>]
 - mvn clean package -DskipTests=true
- Viele Parameter
 - -h (Help)
 - -v (Version; Test Maven Installation)
 - --fail-at-end
 - ...

Build Automation/CI – Beispiel Maven

```
[INFO] Reactor Summary:
[INFO]
[INFO] Activiti ..... SUCCESS [ 0.078 s]
[INFO] Activiti - BPMN Model ..... SUCCESS [ 1.637 s]
[INFO] Activiti - BPMN Converter ..... SUCCESS [ 1.602 s]
[INFO] Activiti - Engine ..... SUCCESS [ 9.965 s]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 13.735 s
[INFO] Finished at: 2018-05-22T16:36:07+02:00
[INFO] Final Memory: 52M/478M
[INFO] -----
```

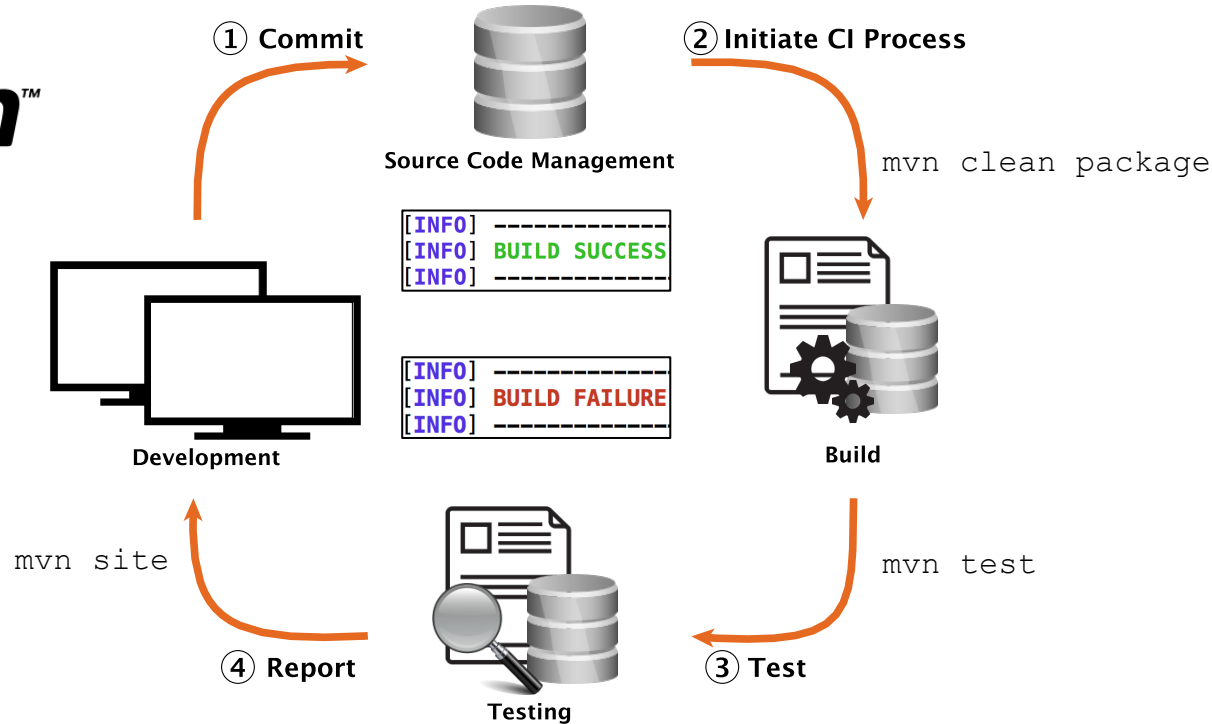
Software Automation Pipeline



Continuous Integration

- Kontinuierliche Integration aller Änderungen (von verschiedenen Entwicklern/Teams)
- Zusammenbauen der Komponenten in eine vollständige Applikation und Ausführen der Integrations Tests mit dieser Applikation

Continuous Integration



Continuous Integration

- Findet meistens auf einem Continuous Integration Server statt

- Tools



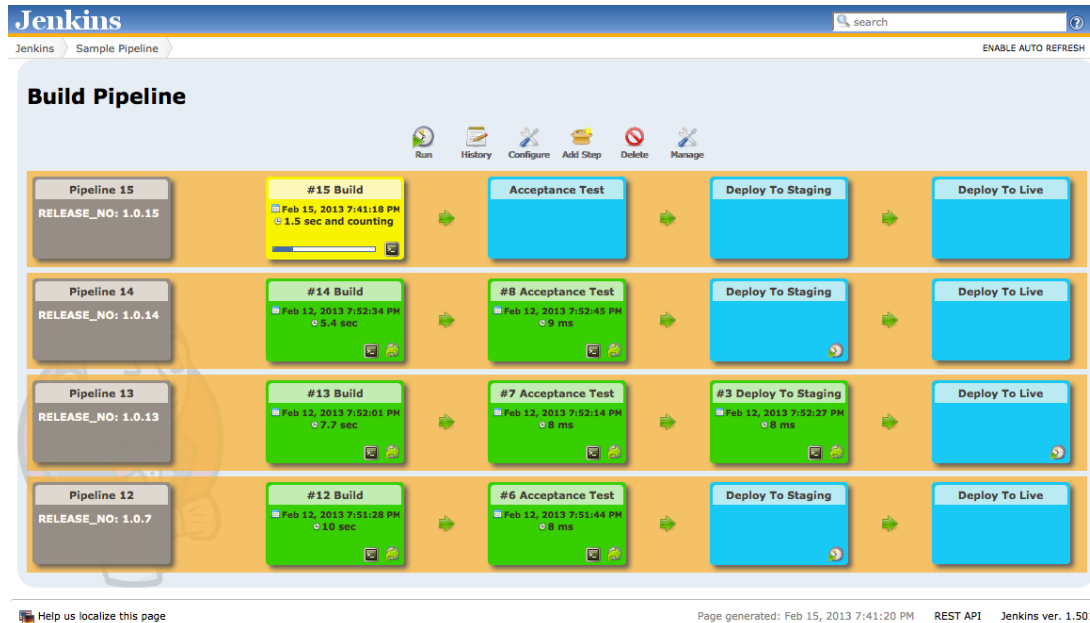
GitHub
Actions



Jenkins

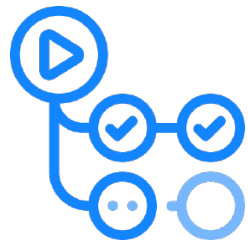


Travis CI

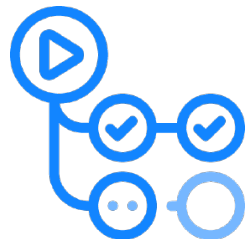


Continuous Integration – GitHub Actions

- Stellt Infrastruktur zur Verfügung
 - Requirements werden deklariert
- Vorteile
 - Standardisiert
 - Geringerer Aufwand (grundsätzlich)
- Nachteile
 - Keine Kontrolle über die dahinter liegende Infrastruktur
 - Customization manchmal schwer



Continuous Integration – GitHub Actions



30 lines (24 sloc) | 656 Bytes

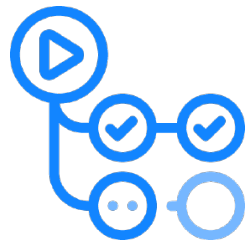
Raw

Blame



```
1
2 name: Android CI
3
4 on:
5   push:
6     branches: [ main ]
7   pull_request:
8     branches: [ main ]
9
10 jobs:
11   build:
12
13     runs-on: ubuntu-latest
14
15     steps:
16     - uses: actions/checkout@v2
17     - name: set up JDK 11
18       uses: actions/setup-java@v2
19       with:
20         java-version: '11'
21         distribution: 'temurin'
22         cache: gradle
23
24     - name: Grant execute permission for gradlew
25       run: chmod +x gradlew
26     - name: Build with Gradle
27       env:
28         GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN } # Needed to get PR information, if any
29         SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
30       run: ./gradlew build jacocoTestReport sonarqube --info
```

Continuous Integration – GitHub Actions



Wann soll der Build
ausgeführt werden?

30 lines (24 sloc) | 656 Bytes

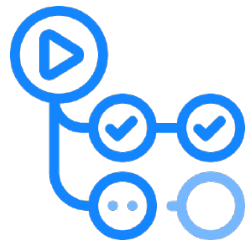
Raw

Blame



```
1
2 name: Android CI
3
4 on:
5   push:
6     branches: [ main ]
7   pull_request:
8     branches: [ main ]
9
10 jobs:
11   build:
12
13     runs-on: ubuntu-latest
14
15     steps:
16     - uses: actions/checkout@v2
17     - name: set up JDK 11
18       uses: actions/setup-java@v2
19       with:
20         java-version: '11'
21         distribution: 'temurin'
22         cache: gradle
23
24     - name: Grant execute permission for gradlew
25       run: chmod +x gradlew
26
27     - name: Build with Gradle
28       env:
29         GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN } # Needed to get PR information, if any
30         SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
31       run: ./gradlew build jacocoTestReport sonarqube --info
```

Continuous Integration – GitHub Actions



30 lines (24 sloc) | 656 Bytes

Raw

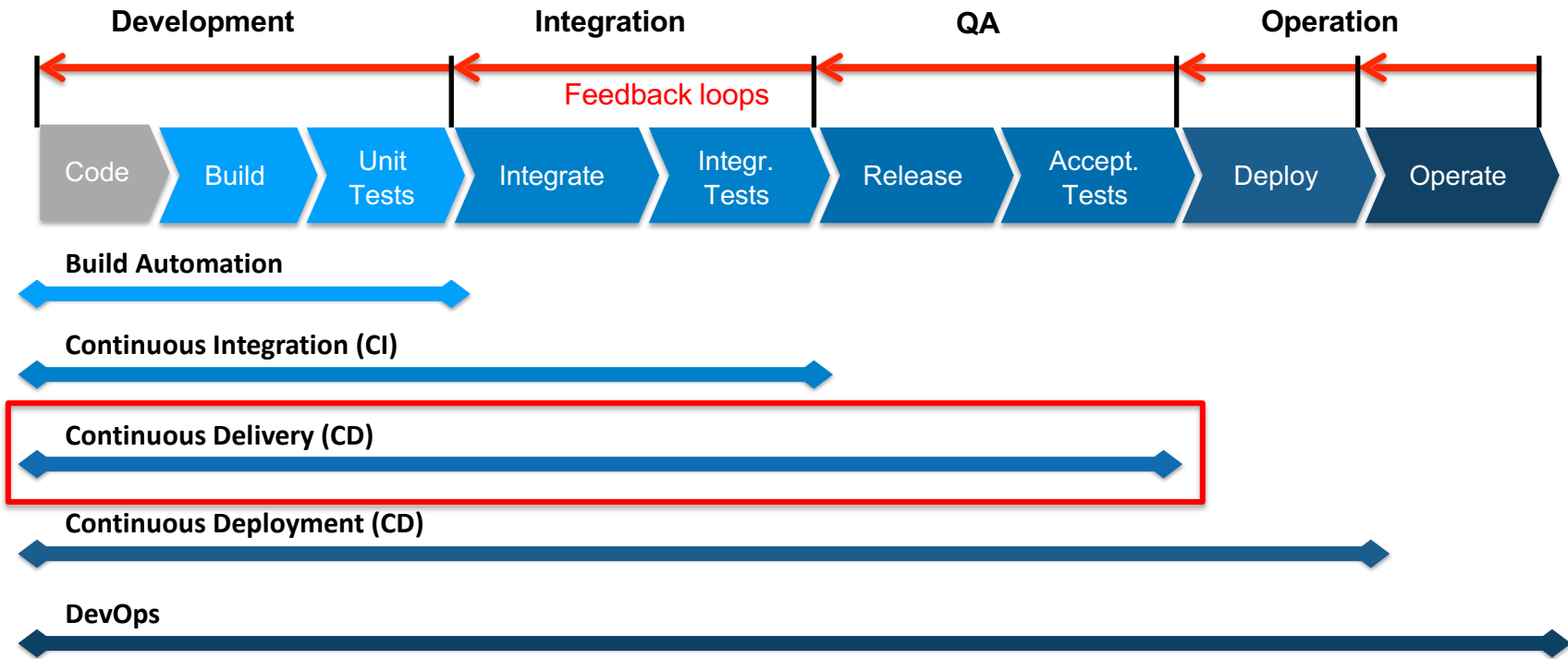
Blame



```
1
2 name: Android CI
3
4 on:
5   push:
6     branches: [ main ]
7   pull_request:
8     branches: [ main ]
9
10 jobs:
11   build:
12
13     runs-on: ubuntu-latest
14
15     steps:
16     - uses: actions/checkout@v2
17     - name: set up JDK 11
18       uses: actions/setup-java@v2
19       with:
20         java-version: '11'
21         distribution: 'temurin'
22         cache: gradle
23
24     - name: Grant execute permission for gradlew
25       run: chmod +x gradlew
26
27     - name: Build with Gradle
28       env:
29         GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN } # Needed to get PR information, if any
30         SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
31       run: ./gradlew build jacocoTestReport sonarqube --info
```

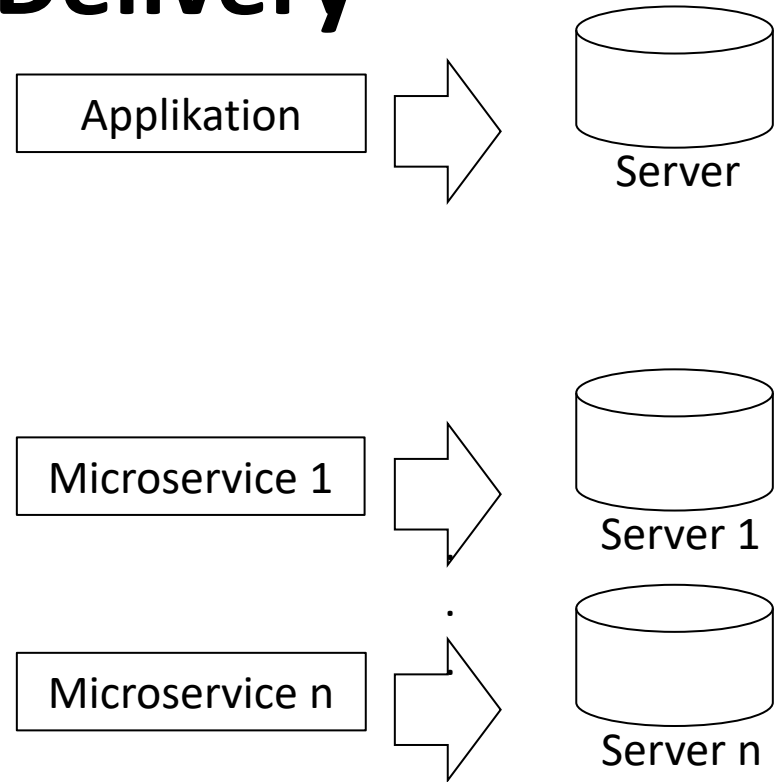
Welche Schritte sollen ausgeführt werden?

Software Automation Pipeline



Continuous Delivery

- Traditionelle Applikationen
 - Monolithen
 - Beim Deployment wird die gesamte Applikation auf eine Server Instanz ausgebracht
 - Meistens wird hier Continuous Integration angewandt
- Flexiblere Architekturen
 - z.B. Microservices
 - Viele kleine Applikationen → viel mehr Deploymentvorgänge



Continuous Delivery

- Schnelles Feedback, ob die Applikation für die Auslieferung bereit ist
- Deployment ist der finale Abschnitt der Continuous Integration
- Ziel: Die Software ist zu jedem Zeitpunkt in einem stabilen Zustand und kann in die Produktionsumgebung ausgebracht werden
- „Definition of done“

Continuous Delivery

- Schnelles Feedback, ob die Applikation für die Auslieferung bereit ist
- Deployment ist der finale Abschnitt der Continuous Integration
- Ziel: Die Software ist zu jedem Zeitpunkt in einem stabilen Zustand und kann in die Produktionsumgebung ausgebracht werden
- „Definition of done“
 - Entwicklung abgeschlossen/Tests erfolgreich/Abnahme durch Kunden

Continuous Delivery

- Schnelles Feedback, ob die Applikation für die Auslieferung bereit ist
- Deployment ist der finale Abschnitt der Continuous Integration
- Ziel: Die Software ist zu jedem Zeitpunkt in einem stabilen Zustand und kann in die Produktionsumgebung ausgebracht werden
- „Definition of done“
 - ~~Entwicklung abgeschlossen/Tests erfolgreich/Abnahme durch Kunden~~
 - Verfügbar in der Produktionsumgebung!

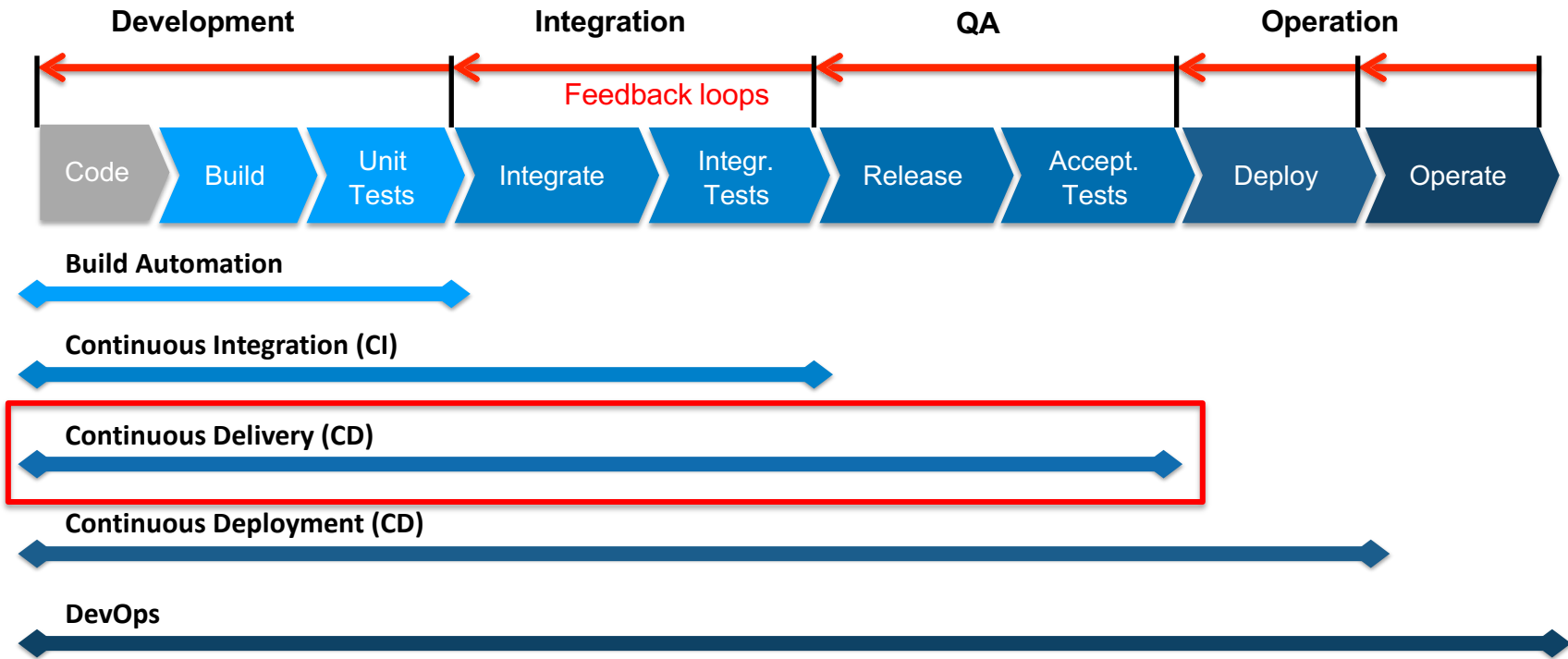
Continuous Delivery

Continuous Delivery ist eine **Sammlung von Prozessen, Tools, und Techniken** um das **Ausliefern des Produkts zu verbessern**

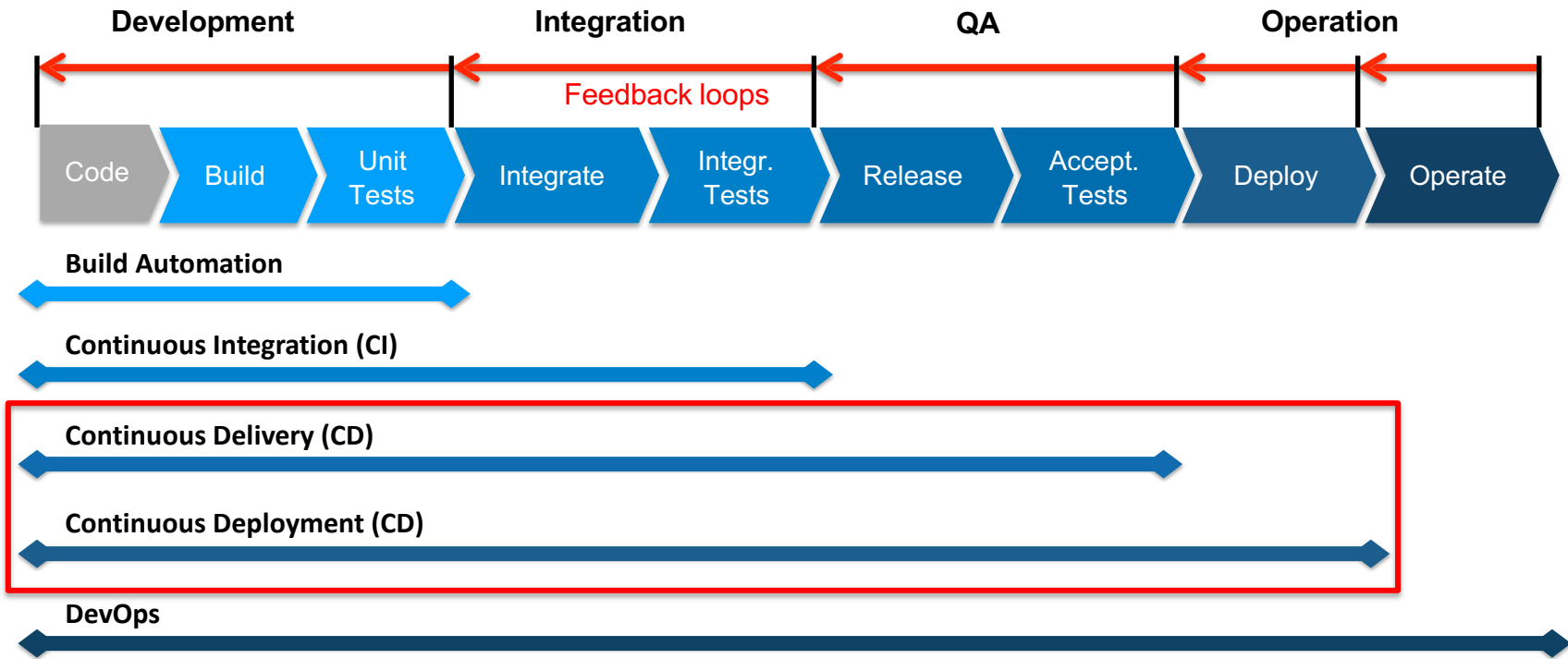
Continuous Delivery

Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.

Software Automation Pipeline



Software Automation Pipeline



Continuous Delivery vs. Deployment

- Bei Continuous Delivery wird der finale Schritt – das Deployment – von einem Menschen ausgelöst.
- Typischer Weise nach einigen Änderungen oder zu fixen Zeitpunkten



- Bei Continuous Deployment wird jede Änderung die erfolgreiche Tests hinter sich hat automatisch in die Produktionsumgebung ausgebracht



Continuous Delivery vs. Deployment

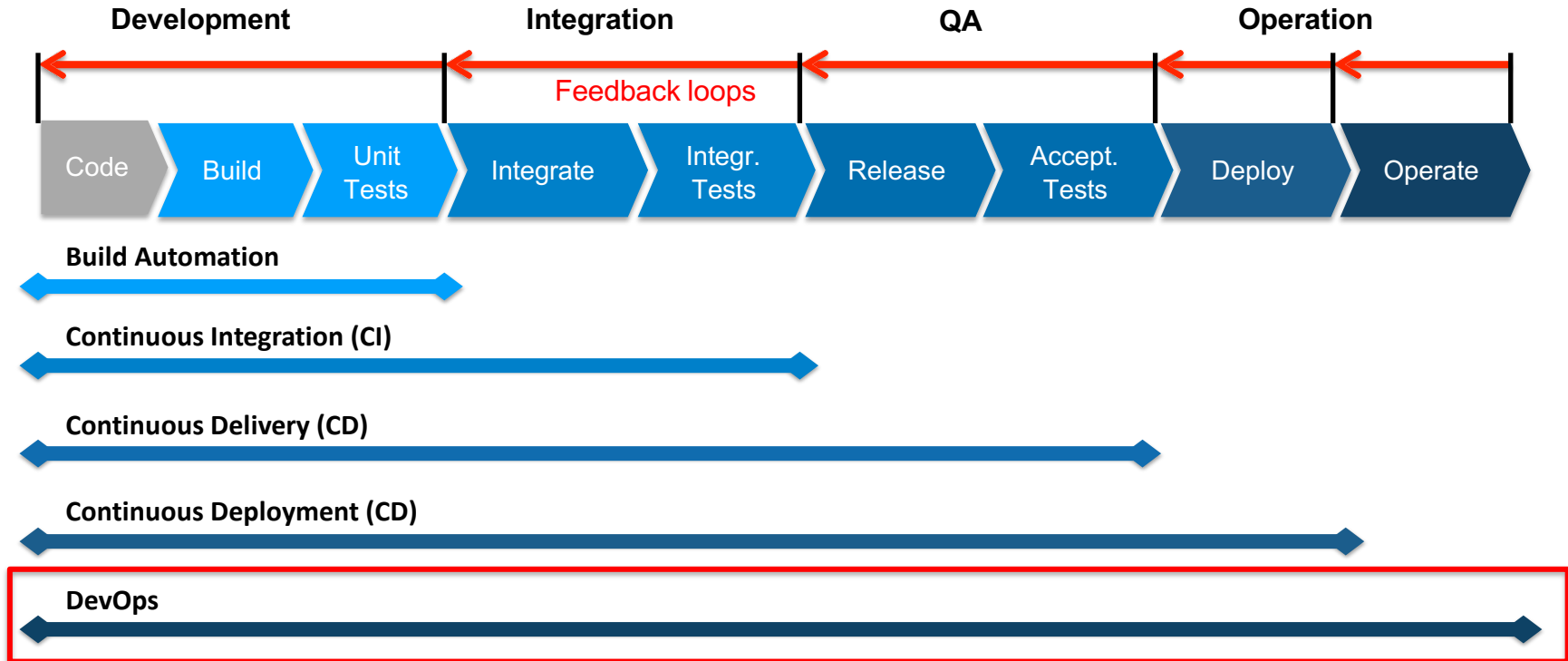
- Bei Continuous Delivery wird der finale Schritt – das Deployment – von einem Menschen manuell gesteuert
- Typischer Vorgehensprozess



- Bei Continuous Deployment wird jede Änderung die erfolgreiche Tests hinter sich hat automatisch in die Produktionsumgebung ausgebracht

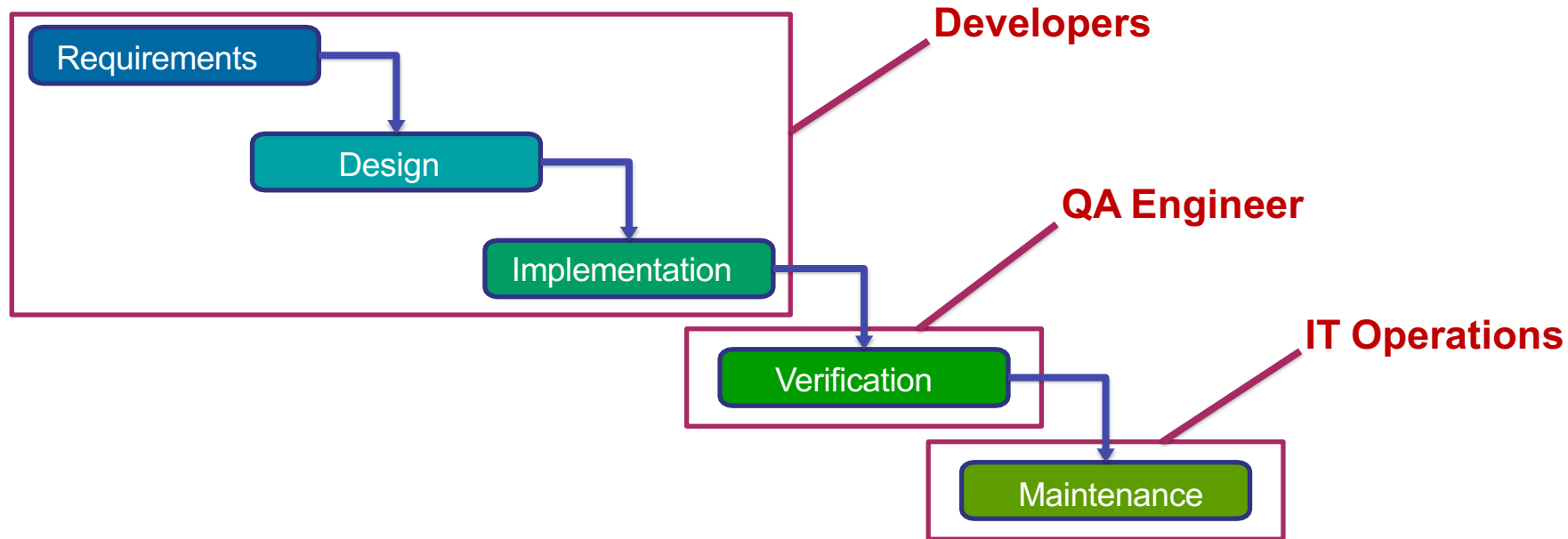


Software Automation Pipeline



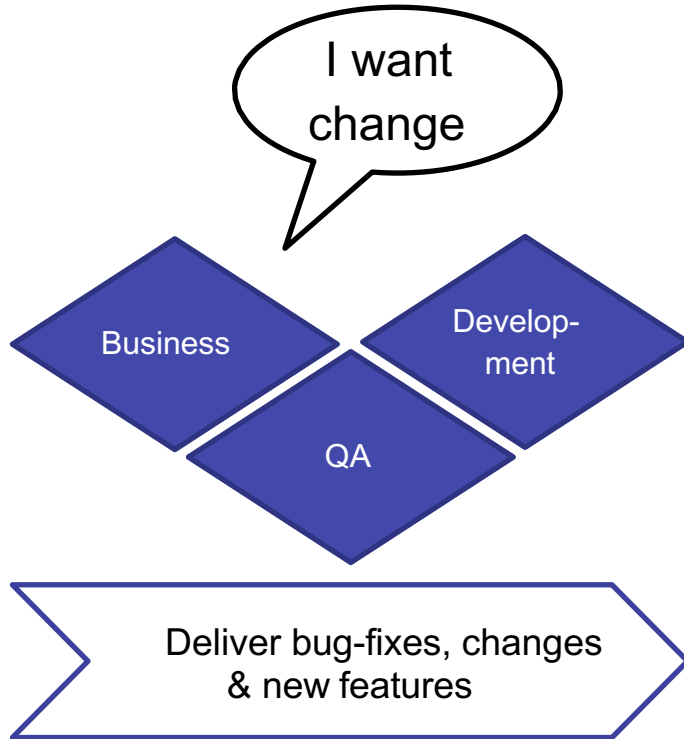
DevOps – Problematik

- Organisationen arbeiten sehr oft noch nach dem Wasserfall – Modell
- Entwicklungsteams arbeiten bereits häufig in agilen Prozessen

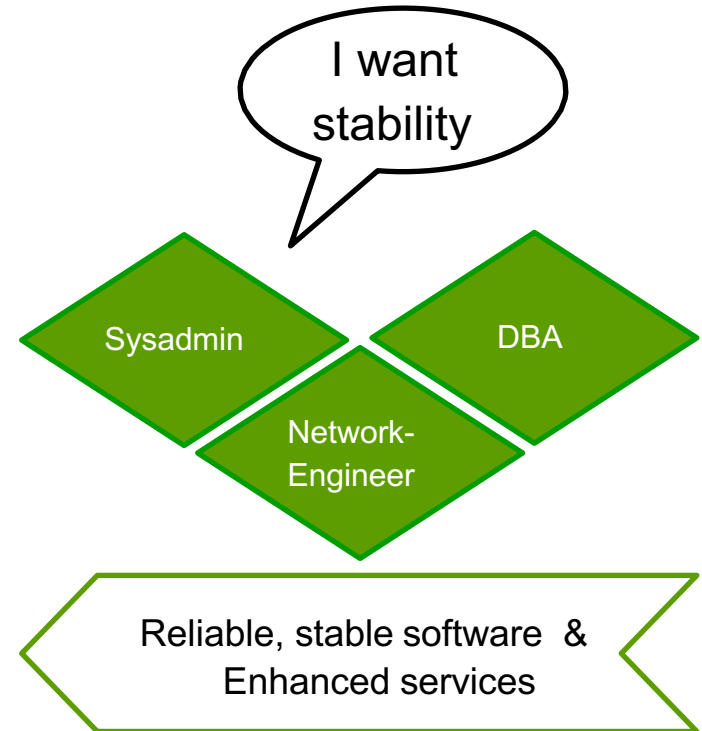
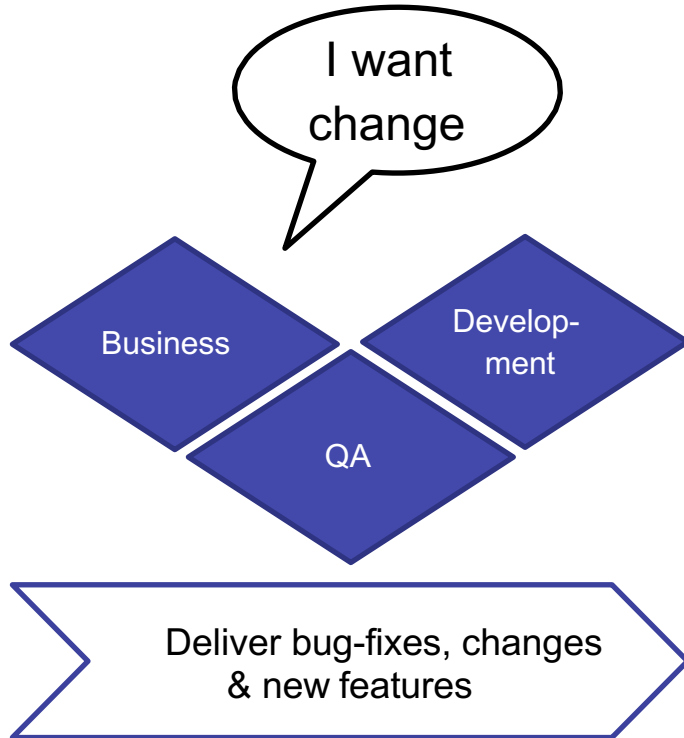


DevOps – Konflikt: „Wall of Confusion“

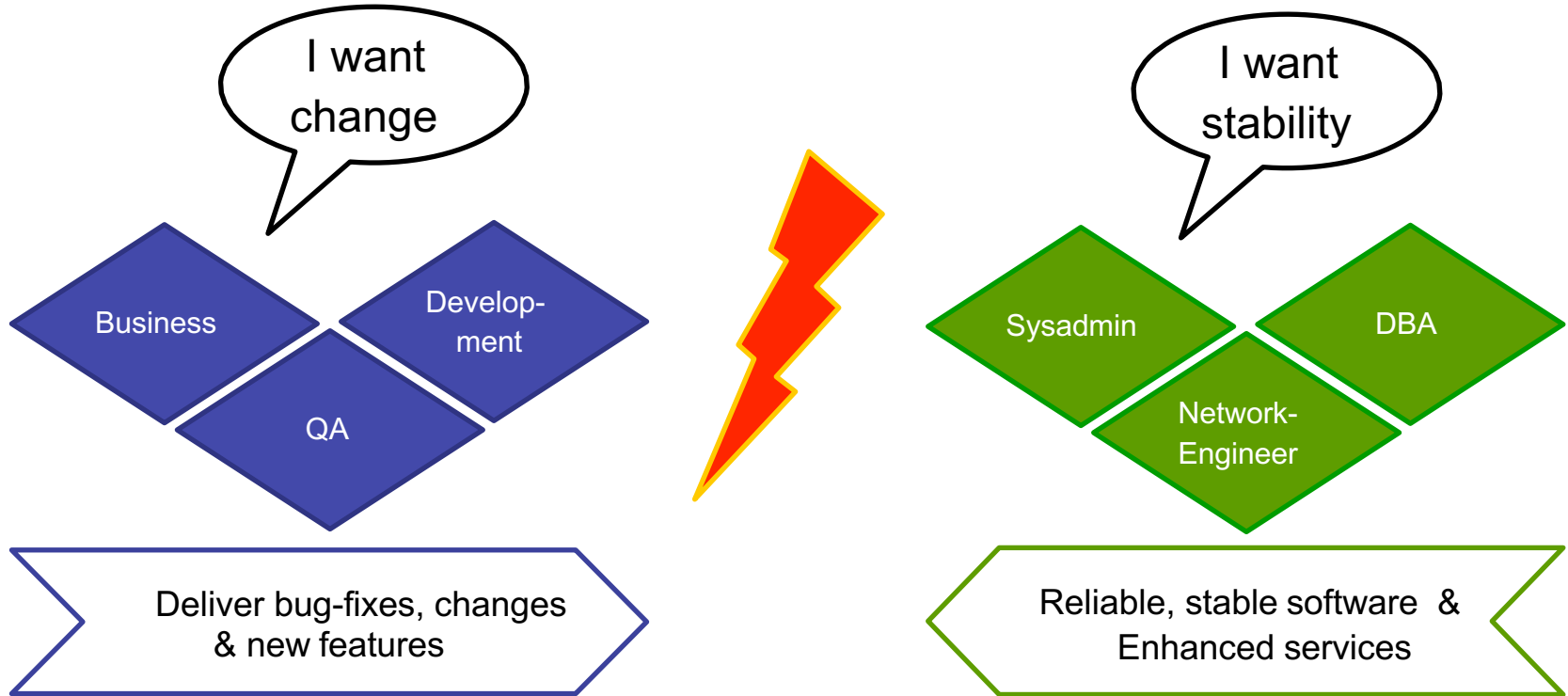
DevOps – Konflikt: „Wall of Confusion“



DevOps – Konflikt: „Wall of Confusion“



DevOps – Konflikt: „Wall of Confusion“



DevOps – Konflikt: „Wall of Confusion“



Spannungsfeld

Entwicklung

- Entwickler bringen nicht immer konsistente Software aus
- Der Entwicklungsprozess ist agil

Betrieb

- Betrieb ist motiviert Änderungen zu widerstehen (Stabilität)
- Der Betriebsprozess ist statisch

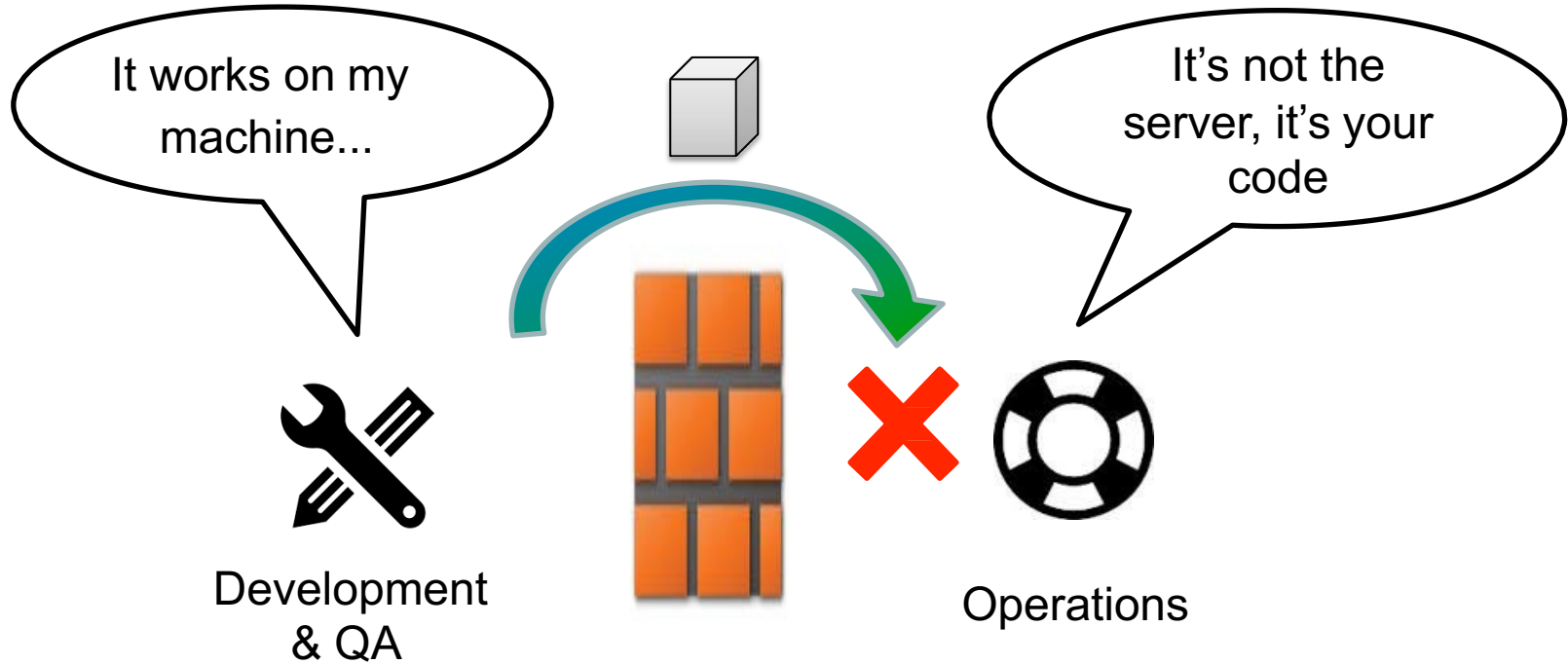
Diese Problematik verursacht Verzögerung und Verzögerungen verursachen Verluste

→ IT ist sehr häufig der Flaschenhals von der Idee zum Markt

**WORKED FINE IN
DEV**

OPS PROBLEM NOW

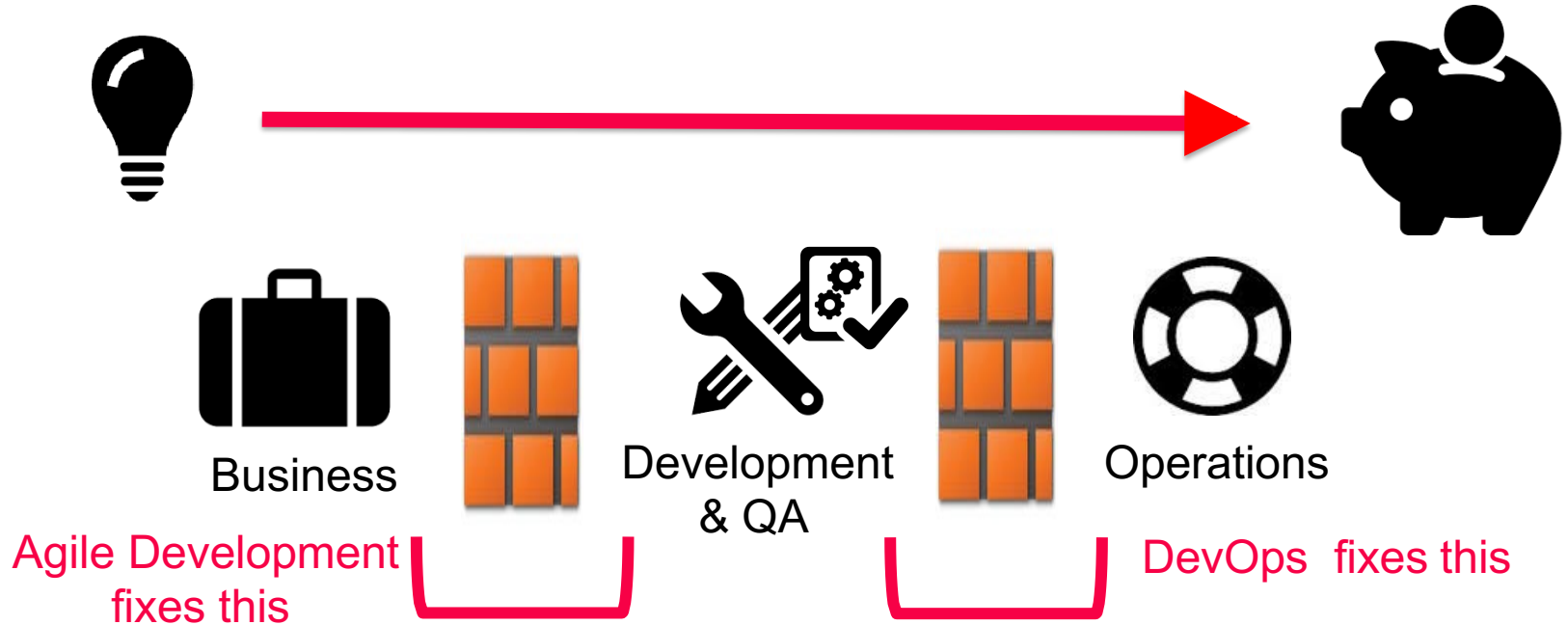
Wall of Confusion



Wall of Confusion - Lösung?



Wall of Confusion - Lösung?



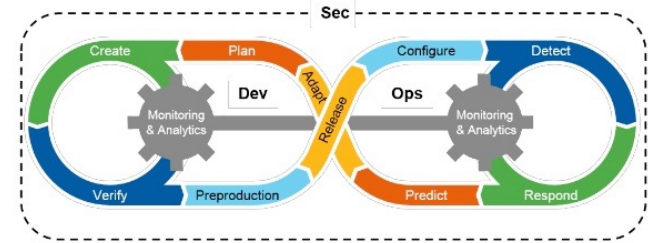
DevOps Practices

- Versionskontrolle für alle Bereiche (auch Betrieb; z.B. für Konfigurationsdateien)
- Automatisiertes Testen
- Proaktives Monitoring (Metriken; auch wenn „alles passt“)
- Anwendung von Kanban/Scrum
- Continuous Integration/Delivery/Deployment
- „Abrufbereitschaft“ auch für Entwicklung
- Virtualisierung/Container (z.B. mit Docker, Kubernetes)

ABER: Es muss auch eine Umstellung der „Kultur“ hinsichtlich der neuen Prozesse und Vorgehensweise erfolgen

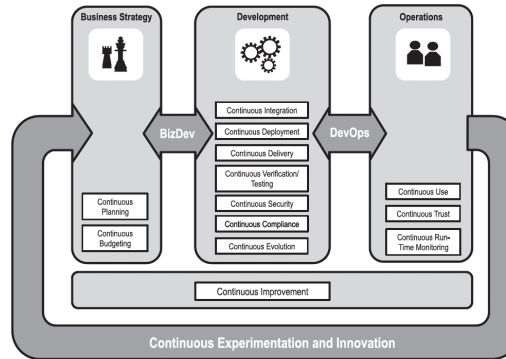
Even with the best tools, DevOps is just another buzzword if you don't have the right culture.

Fortführungen



Source: DevSecOps: How to Seamlessly Integrate Security Into DevOps, Gartner

19



Zusammenfassung

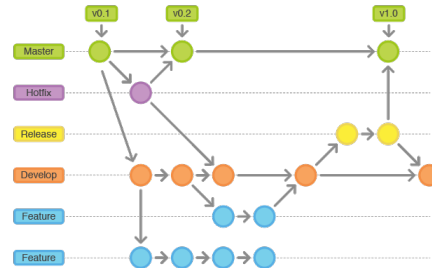
- Build Automation
 - Automatisches Bauen und Testen von Software Produkten
 - Tool Unterstützung z.B. durch Apache Maven
- Continuous Integration/Delivery/Deployment
 - Automatisches Bauen, Testen, und Ausbringen von Software Produkten
 - Tool Unterstützung z.B. durch TravisCI
 - Ziel: Customer Satisfaction
- DevOps
 - Erweiterung von Continuous Deployment um die Operations Seite
 - Erfordert nicht nur gute Tools sondern auch einen Kulturwandel

Aktivitäten/Themen



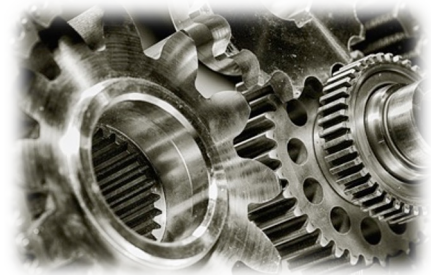
Änderungsmanagement

- Erfassen der Änderungen und Abschätzen der Realisierbarkeit und Kosten
- *Wie gehen SE Prozesse mit Änderungen um?*



Versionsmanagement

- Verwalten der verschiedenen Versionen der System-Komponenten und Vermeidung von Überschneidungen
- *Wie verwaltet man Versionen und Änderungen im Source Code effizient?*



Automatisierung

- Build Management
- Continuous Integration/Delivery/Deployment
- DevOps
- *Wie kann man die Softwareausbringung automatisieren?*

Zum Nachlesen

- [1] Software-Entwurf, Test und Entwicklungsprozess. 8. SW Reuse and CBSE. Roland Mittermeir. 2008.
- [2] Software Engineering, Ian Sommerville. 8th Ed., Addison Wesley, 2007.
- [3] IEEE Standard 828-1990 - IEEE Standard for Software Configuration Management Plan. IEEE Standards Board. Zu finden unter: <http://www.cs.tufts.edu/comp/190/IEEE-std-828-1990-SCM.pdf>
- [4] Software Engineering, Ian Sommerville. 6. Auflage, Pearson Studium, 2001.
- [5] Softwaretechnik: Mit Fallbeispielen aus realen Entwicklungsprojekten., Thomas Grechenig, Mario Bernhart, Roland Breiteneder, Karin Kappel., Pearson Studium. 2010.
- [6] <https://devops.com>
- [7] <https://martinfowler.com/bliki/BlueGreenDeployment.html>
- [8] <https://martinfowler.com/articles/feature-toggles.html>
- [9] <http://cgrant.io/article/deployment-strategies/>
- [10] <https://martinfowler.com/books/continuousDelivery.html>

Weitere Informationen finden Sie auch auf den Web-Sites der Anbieter der diversen Konfigurationsmanagement-Tools